

MATCHI: formal verification of hardware private circuits

Gaëtan Cassiers^{1,2}

¹ CryptoExperts, Paris, France

² UClouvain, Louvain-la-Neuve, Belgium, firstname.lastname@uclouvain.be

Abstract.

Hardware Private Circuits (HPC) provide a compositional framework for designing masked hardware circuits, ensuring security against side-channel attacks in the robust probing model with glitches and transitions. There is however a gap between the simplified circuit models and real-world implementations. In particular, some parts of real circuits are ignored in the simplified models. Further, designing implementations such that they have a simple static correspondence to the theory can be challenging (e.g., around the requirement of splitting the computation into gadgets), or even infeasible (e.g., when some hardware elements like memory are used for different purposes during the executions). In this work, we close this gap by introducing a new model for hardware circuits that include the control and randomness distribution logic. It also allows some computation on the shares to be performed outside the delimited gadgets. We introduce a new open-source compositional masking verification tool, MATCHI. Thanks to the formalization of a composition framework for the new circuit model, we prove that any circuit successfully verified by MATCHI is secure in the threshold probing model with glitches and transitions.

Keywords: Side-channel, Hardware private circuit, PINI, Formal verification

1 Introduction

Masking is a common countermeasure against physical side-channel attacks such as power analysis or electromagnetic analysis attacks. The core principle of masking is to replace computations on secret variables x with computations on randomized sharings $(x_0, \dots, x_d)$ of these variables. With Boolean masking, $x, x_0, \dots, x_d \in \mathbb{F}_{2^k}$ and $x = x_0 \oplus \dots \oplus x_d$, while the sharing $(x_0, \dots, x_d)$ is uniformly distributed over $\mathbb{F}_{2^k}^{d+1}$, conditioned on the previous equality.

The security of masking is often evaluated in the t -probing model. In this model, computation is modeled as an arithmetic circuit and the adversary may observe the values of t arbitrarily-chosen wires in the circuit. A circuit is secure if the view of any such t -probing adversary is statistically independent of the secret variables. When analyzing hardware implementations of masked circuits on ASICs or FPGAs, all variables belong to $\mathbb{F}_2$. Further, the robust t -probing model allows to take into account glitch and transition leakage. These kinds of leakages appear in hardware implementations: transition leakage is caused by the overwriting of values in the circuit from one clock cycle to the next, while glitch leakage generally refers to any leakage that arises from the propagation of values in combinational gates within a clock cycle. These two effects can combine with each other: leakage may depend on both transition and glitches phenomena. Another issue is the lack of composability in the probing model: connecting small t -probing secure circuits (named “gadgets”) does not always result in a t -probing secure circuit. This issue is addressed by composable security properties that imply t -probing security

and (under some conditions) are composable, such as non-interference (NI), strong non-interference (SNI), probe isolating non-interference (PINI) or even output probe isolating non-interference [BBD⁺16, CS20, CS21]. Overall, hardware private circuits (HPC) rely on PINI and OPINI to achieve composable security in the robust probing model with glitches and transitions [CGLS21, CS21].

Therefore, we know in theory how to build securely masked hardware circuits. However, in practice, when building a circuit in a hardware description language (HDL), it is not always clear how to verify that the hypotheses of the composition theorems are satisfied. A first reason is related to the large size of circuits: manually checking properties related to every wire or gadget in the circuit can be tedious and thus prone to mistakes, and the verification of properties that relate multiple elements or clock cycles (e.g., related to the usage of randomness or to transition leakage) is even more challenging to perform by hand. The second reason is the disconnect between theory and practice. For example, theorem assume that the circuit is entirely made of gadgets with sharing connections between them. In practice, we additionally control logic and distribution of randomness to the gadgets, while a lack of abstraction often leads to manipulating arrays of shares instead of abstract sharing objects. Furthermore, manipulating sharings exclusively through black-box gadget instances can be annoying, since it precludes the use of higher level language constructs, for example to handle the control flow of the data. This disconnect reinforces the previous challenges in the verification, leading to little confidence in the overall circuit security. Moreover, models for masked hardware circuits are generally static: each wire in the circuit is statically known to be a share, random, etc. Such a static description may work well in simple cases such as the implementation of a block, however it becomes quite restrictive for more complex use-cases. For example, a memory bus interface for a masked circuit will carry on the same wires some control data, shares, and perhaps even randomness (see [KDT⁺22] for a concrete example). Another example is the implementation of a complex primitive such as ML-KEM [Nat24]. There, a natural implementation for some large linear functions like the number-theoretic transform is share-serial such that all shares of a variable go sequentially on a single wire. It may also happen that, in order to minimize RAM usage, a single byte of memory is used for multiple purposes through the computation.

In this paper, we build a detailed model for masked hardware circuits. This model includes the control and reset logic, as well as the distribution of randomness to the gadgets. Besides this completeness, the model is also very flexible by not statically assigning any role to wires (except for the assumption of synchronous logic with a single clock signal) and instead dynamically assigning them over time, depending on control signals. Using this model, we propose a set of logic rules to decide whether the execution of a circuit is compatible with the requirements of the HPC, and we prove that the rules are sound: any execution accepted by the rules is a secure composition of PINI and OPINI gadgets. In order to allow for flexibility in the implementation, the rules allows manipulation of the shares outside gadgets, under the restriction of these operations being sharewise (which, in practice, restricts such “out of gadget” computations to linear functions on the sharings). We finally design an efficient symbolic circuit simulator that implements the security verification logic and can therefore verify the security of a masked circuit following the HPC paradigm, given a netlist and the values of the circuit inputs. This design is implemented in an open-source tool, MATCHI.¹ To the best of our knowledge, MATCHI is the first tool able to formally prove the security of hardware masked circuits at scale, with the ability to verify a fourth-order implementation of AES in less than a minute.

¹MATCHI (Masking Analyzer for Trivially Composable Hardware Implementations) is available at ANONYMIZED.

2 Background

In this section, we first introduce some basic notations. We then introduce the robust probing model and the composable masking scheme on which we rely in this work. Those explanations remain very high-level since a full formalization adapted to our goals will be given in [Section 3](#).

2.1 Notations and terminology

We use the following notations for integer intervals: $\llbracket a, b \rrbracket = \{a, a+1, \dots, b\}$ and $\llbracket a \rrbracket = \llbracket 1, a \rrbracket$. For tuples, $\parallel$ denotes concatenation.

We use a three-valued simulation for circuits: $\overline{\mathbb{F}_2} = \mathbb{F}_2 \cup \{x\}$, where the symbol x represents an unknown value (this is similar to the x value in Verilog [\[IEE06\]](#)). For any function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$, we define its x -extension $\bar{f} : \overline{\mathbb{F}_2}^n \rightarrow \overline{\mathbb{F}_2}$ as follows. Let $x_1, \dots, x_n \in \overline{\mathbb{F}_2}$, we define $\mathcal{F} = \{f(x'_1, \dots, x'_n) : x'_1 \in e(x_1), \dots, x'_n \in e(x_n)\}$ where $e : \overline{\mathbb{F}_2} \rightarrow \mathcal{P}(\mathbb{F}_2)$ is defined as $e(0) = \{0\}$, $e(1) = \{1\}$ and $e(x) = \{0, 1\}$. Then, $\bar{f}(x_1, \dots, x_n) = e^{-1}(\mathcal{F})$ where e^{-1} is the inverse of e . For vector functions $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^n$, the x -extension is defined from the coordinate-wise x -extensions.

2.2 Robust probing model

The threshold probing model [\[ISW03\]](#) is simple and commonly used to verify the security of masking. A circuit is t -probing secure if, for any tuple of t wires in a circuit, the value of the probes is independent of the secrets represented by the input sharings of the circuit. However, this model does not take into account physical leakage characteristics such as glitches and randomness, which led to the introduction of the robust probing model [\[FGP⁺18\]](#). In this model, each probe observes the value of multiple wires in the circuit. When considering glitches, a probe observes all the wires in the combinational cone that leads to the probed wire. Further, for transitions, a probe observes the value of the wire for two consecutive clock cycles. When combining glitches and transitions a probe observes the values of the inputs of the combinational cone for two consecutive clock cycles (we refer to this model as the “glitch+transition-robust t -probing model”).

2.3 Composable security properties

Since verifying probing security for large circuits is challenging [\[KSM20, BBC⁺19\]](#), a natural approach is the compositional one: building large circuits by connecting small circuits together. These are named “gadgets” and are often assumed to implement the masking simple functions (e.g., logic gates) and to be structured as pipelines. However, probing security and robust probing security are not composable: when connecting (robust) probing secure circuits, it is not guaranteed that the result is (robust) probing secure. This issue is solved by using composable security properties which are stronger than (robust) probing security and composable. The most common composable security properties for masking are based on simulatability, which is a form of probabilistic non-interference [\[BBD⁺16, BBP⁺16\]](#): a tuple of probes in a gadget can be simulated with a subset of the input shares of the gadget if there exists a randomized simulator algorithm taking as input the values of those input shares, whose output is indistinguishable from the true values of the probes.

In particular, we rely on probe-isolating non-interference (PINI) [\[CS20\]](#). With PINI, every share in a $d + 1$ -shares sharing is assigned a unique share index in $\llbracket d + 1 \rrbracket$. Then, a gadget is t -(robust-)PINI if, for any set of (extended) probes P and share indices B with $|P| + |B| \leq t$, the probes in P along with probes on the output shares with index in O can be simulated using the input shares whose index belong $A \cup B$, for some A

satisfying $|A| \leq |P|$. While t -PINI is preserved by arbitrary composition [CS20], as well as glitch-robust t -PINI [CGLS21], these properties are not arbitrarily composable in presence of transitions. This can be solved by using glitch-robust t -output probe isolating non-interferent (OPINI) gadgets, which are arbitrarily composable in the glitch+transition probing model [CS21]. In a nutshell, the difference between PINI and OPINI is that OPINI requires the simulation of all output shares with index in $A \cup B$ (in addition to the probes in P). This stronger requirement means that OPINI gadgets are more significantly more expensive than PINI gadgets, in area, randomness usage and/or latency [CS21, CSV24]. A more efficient solution to composition is proposed in [CS21]: a restricted composition of glitch-robust PINI gadgets. In order to prevent issues caused by transitions, it is sufficient to “reset” the gadget with public inputs between any pair of executions with actual shares as inputs. Further, it is proven that this requirement can be relaxed, in such a way that there is no need for a reset between executions that operate logically in parallel data (i.e., when the inputs of the gadget executions do not depend on the outputs of other executions in the same “non-reset group”).

3 Model

In this section, we introduce our model for hardware circuits. This model is very similar to the one of [CS21]: most notably, it separates the description of the structural circuit (corresponding to a synchronous hardware circuit made of gates and wires) from the execution of this circuit over a sequence of clock cycles (represented by an interval of integers, sometimes simply referred to as “cycle” or “time”). We provide a simple syntactic representation along with semantic evaluation rules which is well-suited to the description of the leakage inference rules in Section 4, unlike the graph-based model of [CS21]. Further, randomness is taken as an input of the circuit and not generated by “random” gates, which is closer to concrete implementations. We then adapt the notions of transition and glitch leakage to this circuit, as well as the (O)PINI definition. Finally, we discuss the modeling of gadgets.

Definition 1 (Structural circuit). A *structural circuit* is a tuple $\mathcal{C} = (\mathcal{W}_\mathcal{C}, \mathcal{A}_\mathcal{C}, \mathcal{I}_\mathcal{C}, \mathcal{O}_\mathcal{C})$ where $\mathcal{W}_\mathcal{C}$ is the set of wires in the circuit, and $\mathcal{A}_\mathcal{C}$ is a set of assignments, while $\mathcal{I}_\mathcal{C}$ and $\mathcal{O}_\mathcal{C}$ are respectively the sets of input and output wires (they are subsets of $\mathcal{W}_\mathcal{C}$). An *assignment* $A \in \mathcal{A}_\mathcal{C}$ is of the form $(y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m)$ where F is a structural gate. The wires $x_1, \dots, x_m \in \mathcal{W}_\mathcal{C}$ are named operands while $y_1, \dots, y_n \in \mathcal{W}_\mathcal{C}$ are the results. A *structural gate* F is characterized by a tuple $(f_F, \ell_F^I, \ell_F^O)$ where $f_F : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^n$ is the function implemented by the gate. The input and output *latency functions* $\ell_F^I : \llbracket n \rrbracket \rightarrow \mathbb{N}$ and $\ell_F^O : \llbracket n \rrbracket \rightarrow \mathbb{N}$ describe the relative latency of the inputs and outputs of the gate.

Further, for a structural circuit $\mathcal{C}$, the wire source function $\text{src}_\mathcal{C} : \mathcal{W}_\mathcal{C} \rightarrow \mathcal{A}_\mathcal{C}$ is defined such that $\text{src}_\mathcal{C}(x) = A$ if there exists an assignment A with x as a result (src is not defined for input wires).

Examples of structural gates The simplest structural gates are the combinational gates, for example in the assignment $y \leftarrow \text{XOR}(x_1, x_2)$ (parentheses are omitted on the left-hand side when there is only one variable), XOR corresponds to the Boolean addition $f_{\text{XOR}}(x_1, x_2) = x_1 \oplus x_2$ and has latency 0: $\ell_{\text{XOR}}^I(1) = \ell_{\text{XOR}}^I(2) = \ell_{\text{XOR}}^O(1) = 0$. Gates for other Boolean functions such as NOT, AND, MUX, ... functions can be similarly defined. We denote by $\mathcal{F}_{\text{comb}}$ the set of combinational gates, that is, all gates F for which $\ell_F^O(i) = 0$ for all $i \in \llbracket n \rrbracket$. Next, the constant 0 (respectively 1) also known as tie low (resp. high) gate corresponds to the assignment $y \leftarrow \text{TieL}()$ (resp. $y \leftarrow \text{TieH}()$) with function $f_{\text{TieL}}() = 0$ (resp. $f_{\text{TieH}}() = 1$) and latency $\ell_{\text{TieL}}^O(1) = \ell_{\text{TieH}}^O(1) = 0$. Finally the register gate (also known

$$\begin{array}{c}
\text{EVALINPUT} \quad \frac{x \in \mathcal{I}_{\mathcal{C}}}{E_{\mathcal{C},f}(x,t) = f(x,t)} \quad \text{EVALGATEEARLY} \quad \frac{(y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \in \mathcal{A}_{\mathcal{C}} \quad t < \ell_F^O(i)}{E_{\mathcal{C},f}(y_j, t) = x} \\
\text{EVALGATE} \quad \frac{(y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \in \mathcal{A}_{\mathcal{C}} \quad \forall i \in \llbracket m \rrbracket, v_i = E_{\mathcal{C},f}(x_i, t + \ell_F^I(i))}{(E_{\mathcal{C},f}(y_1, t + \ell_F^O(1)), \dots, E_{\mathcal{C},f}(y_n, t + \ell_F^O(n))) = \bar{f}_F(v_1, \dots, v_m)} \\
\text{PROBEINPUTG} \quad \frac{x \in \mathcal{I}_{\mathcal{C}}}{P_{\mathcal{C},f}^G(x,t) = f_{\text{glitch}}(x,t)} \quad \text{PROBEREGG} \quad \frac{y \leftarrow \text{Reg}(x) \in \mathcal{A}_{\mathcal{C}}}{P_{\mathcal{C},f}^G(y,t) = E_{\mathcal{C},f_{\text{det}}}(y,t)} \\
\text{PROBECOMBGATEG} \quad \frac{(y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \in \mathcal{A}_{\mathcal{C}} \quad F \in \mathcal{F}_{\text{comb}}}{\forall i \in \llbracket n \rrbracket, P_{\mathcal{C},f}^G(y_i, t) = P_{\mathcal{C},f}^G(x_1, t) \parallel \dots \parallel P_{\mathcal{C},f}^G(x_m, t)} \quad \text{PROBEGT} \quad \frac{x \in \mathcal{W}_{\mathcal{C}}}{P_{\mathcal{C},f}^{GT}(x,t) = P_{\mathcal{C},f}^G(x,t) \parallel P_{\mathcal{C},f}^G(x,t+1)}
\end{array}$$

Figure 1: Circuit and probe valuation rules.

as flip-flop) delays its input by one clock cycle. It appears in assignments $y \leftarrow \text{Reg}(x)$ with $\bar{f}_{\text{Reg}}(x) = x$ and $\ell_{\text{Reg}}^I(1) = 0$, $\ell_{\text{Reg}}^O(1) = 1$.

Definition 2 (Well-formed structural circuit). A structural circuit $\mathcal{C}$ is well-formed under the following conditions. First, each wire $x \in \mathcal{W}_{\mathcal{C}}$ appears either exactly once in the left-hand side of an assignment of the circuit, or belongs to the set of input wires. Second, the circuit cannot contain a combinational loop: a sequence $(x_i)_{i \in \llbracket k \rrbracket}$ such that $x_1 = x_k$ is a combinational loop if for each $i \in \llbracket k-1 \rrbracket$, x_{i+1} is a result of an assignment of which x_i is an operand, and the corresponding input and output latencies of that assignment are equal.

Definition 3 (Well-formed structural gate). A structural gate with m operands and n results F characterized by $(\bar{f}_F, \ell_F^I, \ell_F^O)$ is well-formed if $\ell_F^I(i) \leq \ell_F^O(j)$ for any $i \in \llbracket m \rrbracket$, $j \in \llbracket n \rrbracket$. As a convention, we also assume that $\min(\{\ell_F^I(i) : i \in \llbracket m \rrbracket\} \cup \{\ell_F^O(i) : i \in \llbracket n \rrbracket\}) = 0$.

In the following, we consider exclusively well-wormed circuits. Further, the concrete circuits we work with, named *simple circuits* only contain the gates listed in the previous example: combinational gates and **Reg**. These structural gates are well-formed, which guarantees causality in the sequence of computations.

Circuit execution The semantics of a circuit $\mathcal{C}$ captures its execution over multiple clock cycles. An execution depends on an input valuation function $f : \mathcal{I}_{\mathcal{C}} \times \mathbb{N} \rightarrow \mathbb{F}_2$. The valuation of the execution is the function $E_{\mathcal{C},f} : \mathcal{W}_{\mathcal{C}} \times \mathbb{N} \rightarrow \mathbb{F}_2$. The whole execution valuation is computed by the rules of Figure 1²: where $\bar{f}_F$ is the x -extension of $\bar{f}_F$ (see Section 2). For a well-formed circuit with well-formed gates, the causality and absence of combinational loops guarantee that these rules assign a unique value to each $E_{\mathcal{C},f}(x,t)$.

3.1 Leakage and security

We can now formally define the glitch+transition-robust probing model for our circuits: given a simple structural circuit $\mathcal{C}$, an adversary can put any probe $p \in \mathcal{W}_{\mathcal{C}} \times \mathbb{N}$, and the value observed is given by $P_{\mathcal{C}}^{GT}(p)$, which is defined by inference rules similar to $E_{\mathcal{C},f}$. Contrary to E , $P_{\mathcal{C}}^{GT}$ takes into account glitches on the input wires. Therefore, we use a glitchy input valuation $f : \mathcal{I}_{\mathcal{C}} \times \mathbb{N} \rightarrow \overline{\mathbb{F}_2} \times \overline{\mathbb{F}_2}^*$ where for any x and t , we denote $f(x,t) = (f_{\text{det}}(x,t), f_{\text{glitch}}(x,t))$. The deterministic input valuation is given by f_{det} , while the glitches are given by f_{glitch} (if there are no glitches, $f_{\text{glitch}}(x,t) = f_{\text{det}}(x,t)$), as

²In this notation, all free variables are universally quantified over sets inferred by the domains of the functions/relations in which they appear, the upper part list conditions for the inference rule and the lower parts are the consequences.

described in Figure 1. We also define P^G which is similar to P^{GT} , for the transition-robust probing model. We extend the leakage definitions of leakage to tuples of probes: let $P = ((x_1, t_1), \dots, (x_n, t_n))$, the observed leakage is $P_C^{GT}(P) = P_C^{GT}(x_1, t_1) \parallel \dots \parallel P_C^{GT}(x_n, t_n)$, and likewise for P^G .

A masked circuit is a simple circuit with defined roles for its inputs and outputs. At each clock cycle, each input and output of the circuit can be a public value, a share or a random value. For shares, given the use of the PINI and OPINI notions for security analysis, we only need to know their share index, while other information such as the compositions of sharings is not needed. The possible kinds are represented by the set $\mathcal{K} = \mathcal{K}_p \cup \{\text{rnd}\} \cup \text{sh}(\mathbb{N})$ where $\mathcal{K}_p = \{\text{pub}_v : v \in \overline{\mathbb{F}_2}\}$ and $\text{sh}(S) = \{\text{sh}_i : i \in S\}$ for any $S \subset \mathbb{N}$.

Definition 4 (Masked circuit). A masked circuit a tuple $(\mathcal{C}, \kappa)$ with $\mathcal{C} = (\mathcal{W}_\mathcal{C}, \mathcal{A}_\mathcal{C}, \mathcal{I}_\mathcal{C}, \mathcal{O}_\mathcal{C})$ a simple circuit and $\kappa : (\mathcal{I}_\mathcal{C} \cup \mathcal{O}_\mathcal{C}) \times \mathbb{N} \rightarrow \mathcal{K}$ describes the kinds of the input and output wires, such that $\kappa(x, t) \neq \text{rnd}$ for any $x \in \mathcal{O}_\mathcal{C}$, $t \in \mathbb{N}$.³

This definition of masked circuit is very flexible since it does not hard-code the roles of the input wires. For example, it covers that case of circuits accessed through a bus interface which carries the control data, the secret shares, as well as the randomness. Such a serial interface can be found, e.g., in [KDT⁺22].

When defining the PINI/OPINI security of a masked circuit $(\mathcal{C}, \kappa)$ we have to consider the properties of the input valuations. Indeed, any input valuation considered for an execution input valuations must be compatible with κ .

Definition 5 (Compatible input valuation). An input valuation $f : \mathcal{I}_\mathcal{C} \times \mathbb{N} \rightarrow \overline{\mathbb{F}_2}$ is *compatible* with a masked circuit $(\mathcal{C}, \kappa)$ if for all $x \in \mathcal{I}_\mathcal{C}$ and $t \in \mathbb{N}$, the following conditions hold: (i) if $\kappa(x, t) = \text{pub}_v$ for some $v \in \overline{\mathbb{F}_2}$, then $f(x, t) = v$ and (ii) if $\kappa(x, t) \in \{\text{rnd}\} \cup \text{sh}(\mathbb{N})$, then $f(x, t) \neq x$.

If f is a glitchy input valuation, we say that f is compatible with $(\mathcal{C}, \kappa)$ if f_{det} is compatible with $(\mathcal{C}, \kappa)$. Using compatible input valuations, we may define *correctness* for masked circuits, which is about the public output values.

Definition 6 (Correctness). A masked circuit $(\mathcal{C}, \kappa)$ is correct if, for any compatible input valuation $f : \mathcal{I}_\mathcal{C} \times \mathbb{N} \rightarrow \overline{\mathbb{F}_2}$, for all outputs $x \in \mathcal{O}_\mathcal{C}$ and times $t \in \mathbb{N}$, $E_{\mathcal{C}, f}(x, t) = \kappa(x, t)$.

Further, for the security definition, we have to consider properties of input valuations and relations relationships between them.

Definition 7 (Minimal glitchy input valuation). Let $fg : \mathcal{I}_\mathcal{C} \times \mathbb{N} \rightarrow \overline{\mathbb{F}_2} \times \overline{\mathbb{F}_2}^*$ be a glitchy input valuation compatible with a masked circuit $(\mathcal{C}, \kappa)$. For some $S \subset \mathbb{N}$, f is S -minimal if $f(x, t) = (x, x)$ for any $x \in \mathcal{I}_\mathcal{C}$, $t \in \mathbb{N}$ such that $\kappa(x, t) \in \text{sh}(\mathbb{N} \setminus S) \cup \{\text{rnd}\}$.

Definition 8 (Equivalent glitchy input valuations). Let $f, g : \mathcal{I}_\mathcal{C} \times \mathbb{N} \rightarrow \overline{\mathbb{F}_2} \times \overline{\mathbb{F}_2}^*$ be two glitchy input valuations compatible with a masked circuit $(\mathcal{C}, \kappa)$. For some $Z \subset \mathbb{N}$, f and g are Z -equivalent if $f(x, t) = g(x, t)$ for any $x \in \mathcal{I}_\mathcal{C}$, $t \in \mathbb{N}$ such that $\kappa(x, t) \in \text{sh}(Z)$.

We remark that for any f and Z , there exists a single compatible glitchy input valuation with is Z -equivalent to f and Z -minimal (it is obtained by replacing with (x, x) every $f(x, t)$ not constrained by the compatibility or the f -equivalence). It is denoted as the minimal Z, f -equivalent glitchy input valuation.

Given the masked circuit $(\mathcal{C}, \kappa)$ and a glitchy input valuation f , we can to compute the distributions of inputs with the same share and public values as f (i.e., compatible and

³This restriction is added because masked circuits do not generally serve to produce randomness, hence we do not need to formalize what that would hypothetically mean.

$\mathbb{N}$ -equivalent to f), but with uniform randomness. More formally, the set of glitchy input valuations $\mathcal{R}$ is f -uniform if it follows two conditions. First, any $g \in \mathcal{R}$ is compatible with the masked circuit and $\mathbb{N}$ -equivalent to f . Second, there exists $\rho_{\mathcal{R}} : \mathcal{I}_{\mathcal{C}} \times \mathbb{N} \rightarrow \mathcal{P}(\overline{\mathbb{F}_2} \times \overline{\mathbb{F}_2}^*)$ such that $\mathcal{R} = \{g : \mathcal{I}_{\mathcal{C}} \times \mathbb{N} \rightarrow \overline{\mathbb{F}_2} \times \overline{\mathbb{F}_2}^* \mid \forall x \in \mathcal{I}_{\mathcal{C}}, t \in \mathbb{N} : g(x, t) \in \rho_{\mathcal{R}}(x, t)\}$ and for all $x \in \mathcal{I}_{\mathcal{C}}, t \in \mathbb{N}$ such that if $\kappa(x, t) = \text{rnd}$, then $|\rho_{\mathcal{R}}^0(x, t)| = |\rho_{\mathcal{R}}^1(x, t)|$ where $\rho_{\mathcal{R}}^v(x, t) = \{(\alpha, \beta) \in \rho_{\mathcal{R}}(x, t) \mid \alpha = v\}$. These conditions ensures that uniformly sampling from $\mathcal{R}$ gives a jointly uniform distribution over $\mathbb{F}_2$ for all random inputs under fixed shares and public inputs, and that the glitches on different random inputs are independent of each other.

Let us further denote by $\text{OS}_{(\mathcal{C}, \kappa)} : \mathcal{P}(\mathbb{N}) \rightarrow \mathcal{P}(\mathcal{O}_{\mathcal{C}} \times \mathbb{N})$ the function that computes the set of output shares with given indices: $\text{OS}_{(\mathcal{C}, \kappa, T)}(S) = \{(x, t) \in \mathcal{O}_{\mathcal{C}} \times \llbracket 0, T-1 \rrbracket \mid \kappa(x, t) \in \text{sh}(S)\}$. We can now introduce our definition of simulatability.

Definition 9 (Execution simulation). Let $(\mathcal{C}, \kappa)$ be a masked circuit, $P^{GT}, P^G \in (\mathcal{W}_{\mathcal{C}} \times \mathbb{N})^n$, f be a glitchy input valuation and S a randomized function. The tuples of glitch+transition-extended P^{GT} and glitch-extended probes P^G in and execution with input f are jointly simulated by S if, for any f -uniform set of glitchy input valuations $\mathcal{R}$,

$$(P_{\mathcal{C}, f}^{GT}(P^{GT}), P_{\mathcal{C}, f}^G(P^G)) \approx S(\rho_{\mathcal{R}})$$

where $\approx$ denotes equality of distributions and f is a random variable following a uniform distribution over $\mathcal{R}$.

Definition 10 (I -simulatability). Let $(\mathcal{C}, \kappa)$ be a masked circuit, $P^{GT}, P^G \in (\mathcal{W}_{\mathcal{C}} \times \mathbb{N})^n$ and $I \subset \mathbb{N}$. The tuples glitch+transition-extended P^{GT} and glitch-extended probes P^G are jointly I -simulatable (i.e., it can be simulated using the input share indices I) if there exists a randomized function S such that for any compatible glitchy input valuation f , the probes P^{GT} and P^G are jointly simulated $S(g)$ where g is the minimal $(I \cup O)$, f -equivalent glitchy input valuation.

Definition 11 ((O)PINI (adapted from [CS21])). A masked circuit $(\mathcal{C}, \kappa)$ is glitch+transition-robust d -PINI (respectively d -OPINI) for T clock cycles⁴ if, for any tuple $P \subset \mathcal{W}_{\mathcal{C}} \times \mathbb{N}$ of d_1 extended probes and any set of output shares $O \subset \mathbb{N}$ of size d_2 such that $d_1 + d_2 \leq d$, there exists a set $I \subset \mathbb{N}$ with $|I| \leq d_1$ such that (P, P_O) are jointly $(I \cup O)$ -simulatable with P glitch+transition-extended and P_O glitch-extended⁵, where P_O is an arbitrary-ordered tuple whose elements are $\text{OS}_{\mathcal{C}, \kappa, T}(O)$ for PINI (respectively $\text{OS}_{\mathcal{C}, \kappa, T}(I \cup O)$ for OPINI).

The simulatability definitions and (O)PINI are the similar to [BBP⁺16, CS20, CS21], with adaptation for our new circuit model. In particular, PINI still implies robust probing security: if the input shares belong to uniform $d+1$ -shares sharings of secret variables, then any tuple of d extended probes can be simulated using at most d shares of each input sharing, hence the values of the extended probes are independent of the secrets.⁶

3.2 Gadget composition

We work with simple gadgets such as gadgets implementing a AND gate. On top of being masked circuits, we assume that all gadgets are pipeline circuits (i.e., acyclic circuits whose gates latencies enable a straight-line evaluation).

⁴The limitation to T clock cycles for the output shares serves to bound the size of the set of output shares, due to the definition of leakage: we cannot have leakage tuples of infinite size.

⁵No requiring transition-extended simulation for the output shares is consistent with requesting only the glitches, and not the transitions, on the input wires. This is enough for composition, and does not endanger the probing security, since a probe of P can be put on output wires.

⁶We do not formally define robust d -probing security for our circuit model, since this is not necessary in this work. Such a definition is however straightforwardly derived from [ISW03]: take an *encoding* circuit whose inputs are the secrets and some randomness, and whose outputs are the input shares of $\mathcal{C}$, and adapt Definition 11 to consider secret-equivalent inputs with uniform encoding randomness.

Definition 12 (Pipeline circuit). A structural circuit $\mathcal{C}$ is a pipeline if it does not contain any cycle. That is, there exists an order $\prec$ over its assignments such that for any variable x of $\mathcal{C}$ and any assignments A, A' of $\mathcal{C}$ where x is a result of A and x is an operand of A' , $A \prec A'$. Further, there exists a latency function $\ell : \mathcal{A}_{\mathcal{C}} \cup \mathcal{W}_{\mathcal{C}} \rightarrow \mathbb{N}$ such that (i) $\min_{x \in \mathcal{I}_{\mathcal{C}}} \ell(x) = 0$ and (ii) for any assignment $A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m)$ with $A \in \mathcal{A}_{\mathcal{C}}$, $\ell(x_i) = \ell(A) + \ell_F^I(i)$ for all $i \in \llbracket m \rrbracket$ and $\ell(y_i) = \ell(A) + \ell_F^O(i)$ for all $i \in \llbracket n \rrbracket$. Without loss of generality, we finally assume that $\mathcal{I}_{\mathcal{C}}$ and $\mathcal{O}_{\mathcal{C}}$ are ordered.

For a pipeline circuit $\mathcal{C}$, given an “non-temporal” input valuation $f : \mathcal{I}_{\mathcal{C}} \rightarrow \mathbb{F}_2$, we can define a canonical input valuation $\text{CT}_{f,\ell,x} : \mathcal{I}_{\mathcal{C}} \times \mathbb{N} \rightarrow \mathbb{F}_2$ where, for any f, ℓ and v ,

$$\text{CT}_{f,\ell,v}(x, t) = \begin{cases} f(x) & \text{if } t = \ell(x) \\ v & \text{otherwise} \end{cases}.$$

Similarly, given a non-temporal kind function $\kappa : (\mathcal{I}_{\mathcal{C}} \cup \mathcal{O}_{\mathcal{C}}) \rightarrow \mathcal{K}$, a gadget has a canonical kind function $\text{CT}_{\kappa,\ell,\text{pub}_x}$ and we say that $(\mathcal{C}, \kappa)$ is (canonical) d -(O)PINI if $(\mathcal{C}, \text{CT}_{\kappa,\ell,\text{pub}_x})$ is d -(O)PINI for $\max_{x \in \mathcal{W}_{\mathcal{C}}} \ell(x)$ clock cycles.

Given the orderings $(x_1, \dots, x_m)$ of $\mathcal{I}_{\mathcal{C}}$ and $(y_1, \dots, y_n)$ of $\mathcal{O}_{\mathcal{C}}$, we can define the *functionality* of the pipeline $\mathcal{C}$ as a function $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^n$ such that $f(v_1, \dots, v_m) = (f_1(v_1, \dots, v_m), \dots, f_n(v_1, \dots, v_m))$ with $f_j(v_1, \dots, v_m) = E_{\mathcal{C}, \text{CT}_{\alpha(v_1, \dots, v_m), \ell, x}}(y_j, \ell(y_j))$ and $\alpha(v_1, \dots, v_m)(x_i) = v_i$. The last characteristics of pipeline circuits we need before formally defining gadgets are input and output latency: the input latency of $\mathcal{C}$ is $\ell^I : \llbracket m \rrbracket \rightarrow \mathbb{N}$ with $\ell^I(i) = \ell(x_i)$, and similarly the output latency is $\ell^O : \llbracket n \rrbracket \rightarrow \mathbb{N} : j \mapsto \ell(y_j)$.

Definition 13 (Gadget). A gadget is a pair $\mathcal{G} = (\mathcal{C}, \kappa)$ where $\mathcal{C}$ is a simple pipeline circuit and $\kappa : (\mathcal{I}_{\mathcal{C}} \cup \mathcal{O}_{\mathcal{C}}) \rightarrow \text{sh}(\mathbb{N}) \cup \{\text{rnd}\}$ such that $(\mathcal{C}, \text{CT}_{\kappa,\ell,\text{pub}_x})$ is a masked circuit. Further, the “gadget gate” corresponding to $\mathcal{G}$ is a structural gate $F_{\mathcal{G}}$ with characteristic $(f_{\mathcal{G}}, \ell_{\mathcal{G}}^I, \ell_{\mathcal{G}}^O)$, where $f_{\mathcal{G}}$ is the functionality of the pipeline $\mathcal{C}$, while $\ell_{\mathcal{G}}^I$ and $\ell_{\mathcal{G}}^O$ are the input and output latencies of the pipeline $\mathcal{C}$. The gadget gate is further associated with the input and output kind functions $\kappa_{F_{\mathcal{G}}}^I$ and $\kappa_{F_{\mathcal{G}}}^O$ where $\kappa_{F_{\mathcal{G}}}^I : \llbracket m \rrbracket \rightarrow \mathcal{K} : i \mapsto \kappa(x_i)$ and $\kappa_{F_{\mathcal{G}}}^O : \llbracket n \rrbracket \rightarrow \mathcal{K} : j \mapsto \kappa(y_j)$.

For the composition of gadgets, we define *extended* circuits, which are structural circuits where some gates are gadget gates (which belong all to a set $\mathcal{F}_{\text{gadget}}$), while other gates are combinational gates (in $\mathcal{F}_{\text{comb}}$) and **Reg.**⁷ Given an extended circuit $\mathcal{C}$, its expansion $\mathcal{E}_{\mathcal{C}}$ is a simple circuit that performs the same computations. It is obtained by replacing each gadget gate with the pipeline circuit of the corresponding gadget (in this replacement, the wires of the pipeline circuit are replaced: input and output wires become the operands and results of the gadget gate, while the other wires are instantiated with new and unique wires).

Let us provide a way to identify assignments originating from gadget expansion in $\mathcal{E}_{\mathcal{C}}$. Let $A \in \mathcal{A}_{\mathcal{C}}$ be an assignment with a gadget gate and $\mathcal{G} = (\mathcal{C}_{\mathcal{G}}, \kappa_{\mathcal{G}})$ be the corresponding gadget. For any $A' \in \mathcal{A}_{\mathcal{C}_{\mathcal{G}}}$ there is an assignment in $\mathcal{A}_{\mathcal{E}_{\mathcal{C}}}$ corresponding to A and A' denoted $\mathcal{E}_{\mathcal{C}}^{A,A'}$ and for any $x' \in \mathcal{W}_{\mathcal{C}_{\mathcal{G}}}$ there is a wire in $\mathcal{W}_{\mathcal{E}_{\mathcal{C}}}$ denoted $\mathcal{E}_{\mathcal{C}}^{A,x'}$. We also use the notation $\mathcal{E}_{\mathcal{C}}^A = \{\mathcal{E}_{\mathcal{C}}^{A,A'} : A' \in \mathcal{A}_{\mathcal{C}_{\mathcal{G}}}\} \cup \{\mathcal{E}_{\mathcal{C}}^{A,x} : x \in \mathcal{W}_{\mathcal{C}_{\mathcal{G}}}\}$. This notation is extended to consider timing: given the latency function ℓ in the pipeline definition of $\mathcal{G}$, $\mathcal{E}_{\mathcal{C},t}^{A,A'} = (\mathcal{E}_{\mathcal{C}}^{A,A'}, t + \ell(A'))$ and $\mathcal{E}_{\mathcal{C},t}^{A,x} = (\mathcal{E}_{\mathcal{C}}^{A,x}, t + \ell(x))$, as well as $\mathcal{E}_{\mathcal{C},t}^A = \{\mathcal{E}_{\mathcal{C},t}^{A,A'} : A' \in \mathcal{A}_{\mathcal{C}_{\mathcal{G}}}\} \cup \{\mathcal{E}_{\mathcal{C},t}^{A,x} : x \in \mathcal{W}_{\mathcal{C}_{\mathcal{G}}}\}$. We also define a function $S_{\mathcal{C}}^W$ to map wires from the expanded circuit back to the wires in the gadget: $S_{\mathcal{C}}^W(x) = x'$ for any $x \in \mathcal{W}_{\mathcal{E}_{\mathcal{C}}}$ and $x' \in \mathcal{W}_{\mathcal{C}_{\mathcal{G}}}$ such that $\mathcal{E}_{\mathcal{C}}^{A,x'} = x$ for some $A \in \mathcal{A}_{\mathcal{C}}$.

⁷While extended circuits should remain well-formed, the gates that represent gadgets are not necessarily well-formed: they may have outputs at different latencies, possibly producing outputs before the last input is provided.

$$\begin{array}{c}
\frac{\text{VALUESEARLY} \quad y \leftarrow \text{Reg}(x) \in \mathcal{A}_{\mathcal{E}_{\mathcal{C}}} \quad V(y[0]) = x}{V(y[0]) = x} \quad \frac{\text{VALUESPUB} \quad x \in \mathcal{I}_{\mathcal{C}} \quad \kappa(x, t) = \text{pub}_v \quad V(x[t]) = v}{V(x[t]) = v} \quad \frac{\text{VALUESRND} \quad x \in \mathcal{I}_{\mathcal{C}} \quad \kappa(x, t) = \text{rnd} \quad V(x[t]) = x}{V(x[t]) = x} \\
\\
\frac{\text{VALUESSHARE} \quad x \in \mathcal{I}_{\mathcal{C}} \quad \kappa(x, t) = \text{sh}_i \quad V(x[t]) = x}{V(x[t]) = x} \quad \frac{\text{VALUESREG} \quad y \leftarrow \text{Reg}(x) \in \mathcal{A}_{\mathcal{E}_{\mathcal{C}}} \quad V(y[t+1]) = V(x[t])}{V(y[t+1]) = V(x[t])} \quad \frac{\text{VALUESCOMB} \quad y \leftarrow F(x_1, \dots, x_m) \in \mathcal{A}_{\mathcal{E}_{\mathcal{C}}} \quad F \in \mathcal{F}_{\text{comb}} \quad V(y[t]) = \overline{F}(V(x_1[t]), \dots, V(x_m[t]))}{V(y[t]) = \overline{F}(V(x_1[t]), \dots, V(x_m[t]))} \\
\\
\frac{\text{INCORRECTOUTPUT} \quad x \in \mathcal{O}_{\mathcal{C}} \quad \kappa(x, t) = \text{pub}_v \quad v \in \{0, 1\} \quad V(x[t]) \neq v}{(\mathcal{C}, \kappa) : \text{Correct}}
\end{array}$$

Figure 2: Wire value inference rules.

A masked extended circuit is a pair $(\mathcal{C}, \kappa)$ where $\mathcal{C}$ is an extended circuit and such that $(\mathcal{E}_{\mathcal{C}}, \kappa)$ is a masked circuit. Moreover, we say that $(\mathcal{C}, \kappa)$ is d -(O)PINI for T clock cycles if $(\mathcal{E}_{\mathcal{C}}, \kappa)$ is d -(O)PINI for T clock cycles. Correctness is defined similarly: $(\mathcal{C}, \kappa)$ is correct if $(\mathcal{E}_{\mathcal{C}}, \kappa)$ is correct.

When using MATCHI, PINI and OPINI gadgets will be specifically-annotated netlist modules (formally, we assume that all gates corresponding to PINI (but not OPINI) gadgets belong to $\mathcal{F}_{\text{PINI}}$ and the ones for OPINI gadgets belong to $\mathcal{F}_{\text{OPINI}}$, with $\mathcal{F}_{\text{gadget}} = \mathcal{F}_{\text{PINI}} \cup \mathcal{F}_{\text{OPINI}}$). However, we do not require all operations on shares to be done in such modules: operations on a single share index (including all “sharewise gadgets”) can be performed as “plain” gates in the extended circuit. Only operations whose security is complex to verify, such as AND gadgets, needs to be encapsulated within netlist modules annotated as gadgets.

4 Leakage inference

In this section, we introduce inference rules for modeling the leakage. Starting from an extended masked circuit $(\mathcal{C}, \kappa)$, these rules describe how we infer that $\mathcal{C}$ is d -PINI. Actually, the inference detects violations of a composition policy, and in such cases it concludes $(\mathcal{C}, \kappa) : \overline{\text{PINI}}$. After describing the inference rules, we then prove that if that conclusion cannot be reached (i.e., $\neg((\mathcal{C}, \kappa) : \overline{\text{PINI}})$ ⁸) and if all gadgets corresponding to gates in $\mathcal{F}_{\text{PINI}}$ (respectively $\mathcal{F}_{\text{OPINI}}$) are d -(O)PINI, then $(\mathcal{C}, \kappa)$ is d -PINI.

4.1 Inference rules

We group the presentation of the inference rules in sets that model different characteristics of the masked circuit. These rules mainly derive properties of wires in the circuit, for each clock cycle. First, the set of possible values for the wires. Then, the dependency of each wire (including glitches) on shares of each index is tracked, along with the PINI gadget execution these shares originate from. Next, the possible policy violations due to transitions between executions of PINI gadgets are analyzed. The following analysis set is focused on the random values: how they are used by gadgets and where they leak. Finally, the properties of the inputs (shares and randoms) of (O)PINI gadgets and of the circuit’s output shares are verified.

Wire values The computation of the possible wire values uses the expanded circuit $\mathcal{E}_{\mathcal{C}}$ and applies the rules of Figure 2, which are the same as the circuit execution rules in Section 3, with the important difference that all non-public inputs are assumed to be unknown, i.e., x (which means “either 0 or 1”). For any $x \in \mathcal{W}_{\mathcal{E}_{\mathcal{C}}}$, $t \in \mathbb{N}$ and $v \in \overline{\mathbb{F}}_2$, we

⁸Where $\neg(X)$ is the weak negation of X , i.e., it means “we cannot conclude X from the rules”.

$$\begin{array}{c}
\text{DEPPUB} \quad \frac{x \in \mathcal{I}_C \quad \kappa(x, t) = \text{pub}_v}{D(x[t]) = GD(x[t]) = \emptyset} \quad \text{DEPRND} \quad \frac{x \in \mathcal{I}_C \quad \kappa(x, t) = \text{rnd}}{D(x[t]) = GD(x[t]) = \{\text{rnd}_{(x,t)}\}} \quad \text{DEPSHARE} \quad \frac{x \in \mathcal{I}_C \quad \kappa(x, t) = \text{sh}_i}{D(x[t]) = \{\text{sh}_i\} \quad GD(x[t]) = \{\text{sh}_i\}} \\
\text{PGADGET} \quad \frac{A \in \mathcal{A}_C \quad A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \quad F \in \mathcal{F}_{\text{gadget}} \quad i \in \llbracket m \rrbracket \quad \text{sh}_j \in GD(x_i[t + \ell_F^I(i)])}{A : \text{PG}(t)} \quad \text{NPGADGET} \quad \frac{\neg(A : \text{PG}(t))}{A : \overline{\text{PG}}(t)} \\
\text{GLREG} \quad \frac{A \equiv y \leftarrow \text{Reg}(x) \quad A \in \mathcal{A}_C}{A : \text{GLA}(t)} \quad \text{GLCOMB} \quad \frac{A \equiv y \leftarrow F(x_1, \dots, x_m) \quad A \in \mathcal{A}_C \quad F \in \mathcal{F}_{\text{comb}}}{A : \text{GLA}(t)} \quad \text{GLDUMMYGADGET} \quad \frac{A' \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \quad A \in \mathcal{A}_C \quad A : \text{PG}(t) \quad (A', t') \in \mathcal{E}_{C,t}^A}{A' : \text{GLA}(t')} \\
\text{DEPREG} \quad \frac{A \equiv y \leftarrow \text{Reg}(x) \quad A : \text{GLA}(t)}{D(y[t+1]) = D(x[t]) \quad GD(y[t+1]) = D(x[t])} \quad \text{DEPCOMBV} \quad \frac{A \equiv y \leftarrow F(x_1, \dots, x_m) \quad A : \text{GLA}(t) \quad F \in \mathcal{F}_{\text{comb}} \quad V(y[t]) \in \{0, 1\}}{D(y[t]) = \emptyset} \\
\text{DEPCOMBX} \quad \frac{A \equiv y \leftarrow F(x_1, \dots, x_m) \quad A : \text{GLA}(t) \quad F \in \mathcal{F}_{\text{comb}} \quad V(y[t]) = x}{D(y[t]) = \bigcup_{i=1}^m D(x_i[t])} \quad \text{GLITCHDEPCOMB} \quad \frac{A \equiv y \leftarrow F(x_1, \dots, x_m) \quad A : \text{GLA}(t) \quad F \in \mathcal{F}_{\text{comb}}}{GD(y[t]) = \bigcup_{i=1}^m GD(x_i[t])} \\
\text{DEPPGADGET} \quad \frac{A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \quad A \in \mathcal{A}_C \quad F \in \mathcal{F}_{\text{gadget}} \quad A : \text{PG}(t) \quad \kappa_F^O(j) = \text{sh}_i}{D(y_j[t + \ell_F^O(j)]) = GD(y_j[t + \ell_F^O(j)]) = \{\text{sh}_i\}} \\
\text{GLITCHTRANSLA} \quad \frac{x \in \mathcal{W}_C}{\text{GTE}(GD(x[t]) \cup GD(x[t+1]))} \quad \text{FAILDEPSHARES} \quad \frac{\text{GTE}(S) \quad \text{sh}_i, \text{sh}_j \in S \quad i \neq j}{(\mathcal{C}, \kappa) : \text{PINI}}
\end{array}$$

Figure 3: Share dependency inference rules.

denote $v = V(x[t])^9$ when the possible values of x at time t are described by v . In other words, $v = V(x[t])$ is equivalent to $E_{\mathcal{E}_C, f}(x, t) = v$ where $f(x, t) = v$ if $\kappa(x, t) = \text{pub}_v$ and $f(x, t) = x$ otherwise.

As an aside, we introduce a rule to detect correctness issues in the output public values, as it is an interesting property to verify, even though it is not directly related to side-channel security.

Dependencies on shares and randomness For each clock cycle, we determine which secret shares each wire depends on, for both the deterministic value carried by the wires, and for possible glitches. For any $x \in \mathcal{W}_C$ and $t \in \mathbb{N}$, if the wire x at cycle t depends on a share index i , then $\text{sh}_i \in D(x[t])$. We also track dependencies on input randomness: for each input random wire y and time t' , if x at cycle t depends on $y[t']$, then $\text{rnd}_{(y,t')} \in D(x[t])$. Similarly, $GD(x[t])$ denotes the dependencies in the glitch domain.

In Figure 3, we first describe the dependencies for input wires, which is quite straightforward: input shares naturally depend on a share index, and random inputs depend on themselves; the glitch domain is identical to the non-glitch domain. Next, we consider gadgets, for which there are two cases. If none of the inputs depends on any share, the execution of the gadget is “dummy” and we simply treat its gates like any other gates in the circuit. Otherwise, we have a “proper” execution of the gadget, with constraints on the input wires (that we will check in Section 4.1), thus the outputs are “new” sharings, with no dependency except their share indices. An execution of an assignment A with a gadget gate is proper, denoted $A : \text{PG}(t)$, if any of its input is glitch-dependent on an input share. We denote by $A : \overline{\text{PG}}(t)$ the dummy gadget assignment executions. These two cases for the dependency analysis materialize when considering assignments in $\mathcal{E}_C$:

⁹ $x[t]$ is a symbol that informally means “ x at clock cycle t ” but does not evaluate down to another value.

$$\begin{array}{c}
\text{DPGEPINIGADGET} \\
\frac{A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \quad A \in \mathcal{A}_C \quad F \in \mathcal{F}_{\text{PINI}} \quad A : \text{PG}(t)}{\forall i \in \llbracket n \rrbracket, y_i[t + \ell_F^O(i)] : \text{DPGE}(t, A)} \\
\text{FAILDPGE} \\
\frac{A \in \mathcal{A}_C \quad F \in \mathcal{F}_{\text{PINI}} \quad A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \quad x_i[t + \ell_F^I(i)] : \text{DPGE}(t', A) \quad \forall t'' \in \{t', \dots, t\}, A : \text{PG}(t'')}{(\mathcal{C}, \kappa) : \overline{\text{PINI}}}
\end{array}
\quad
\begin{array}{c}
\text{DPGEASSIGN} \\
\frac{(y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \in \mathcal{A}_C \quad x_i[t + \ell_F^I(i)] : \text{DPGE}(t', A) \quad \forall (y_j[t + \ell_F^O(j)]) = x}{y_j[t + \ell_F^O(j)] : \text{DPGE}(t', A)}
\end{array}$$

Figure 4: Analysis of transitions between PINI gadget executions.

the execution of assignments that correspond to a PG gadget and the ones that do not (either not in a gadget, or in a $\overline{\text{PG}}$ gadget execution). We denote $A : \text{GLA}(t)$ (“Gate-Level Analysis”) the latter case.

We now discuss gate-level dependency inference for all assignments $A : \text{GLA}(t)$. Since register copy their input value while stopping glitches, their output D and GD are the same as their input D. For combinational gates, the propagation of glitch dependencies is trivial: the glitch dependencies on the output is the union of all the ones in the inputs. The inference for non-glitch dependencies is more complex, as it depends on the input dependencies, but also on the functionality of the gate and the possible values of the inputs. For example, in $y \leftarrow \text{AND}(x_1, x_2)$ where $V(x_1[t]) = 0$, we can infer $D(y) = \emptyset$. This explains the two rules DEPCOMBV and DEPCOMBX . For assignments which are PG gadget executions, we consider the dependencies in their inputs to be “terminal”: we will check in Section 4.1 that the inputs satisfy the constraints of the gadget compositions. Therefore, the output shares of the gadget are “fresh” sharings, with no dependency other than themselves.

Finally, if any wire at any clock cycle is (glitch-)dependent on more than one share index, the whole circuit cannot be PINI. Considering transitions, we should also consider two consecutive clock cycles, as formalized in the following rules where we first define the glitch+transition-extended probes GTE.

Transitions between PINI gadget executions The secure composition of PINI gadgets with transitions explained in Section 2.3 is formalized in Figure 4: it requires that if any input of the execution of a gadget assignment A at a cycle t depends on an output of the same assignment at cycle $t' < t$, then there must be a t'' such that $t' < t'' < t$ where the inputs of A do not (glitch-)depend on any share index. In order to verify this property, we track for each wire and each clock cycle the gadget assignments it depends on, formalized as $x[t] : \text{DPGE}(t', A)$ (“Depends on PINI Gadget Execution”). The verification then checks whether an input of a PINI gadget depends on a previous execution of the same gadget assignment, with all executions in-between being PG.

Randomness We verify in Figure 5 that the randomness inputs of PG gadgets is uniformly distributed and fresh, that is, that it does not leak outside the gadget. However, we allow some leakage in order to enable control logic to distribute or store it. For this purpose, we first identify which wires are equal to (or in bijection with) an input random value. This is trivial for input wires and Reg. For combinational gates, we first need to define whether the gate is bijective. The following rules use the function $e : \overline{\mathbb{F}_2} \rightarrow \mathcal{P}(\mathbb{F}_2)$ defined as $e(0) = \{0\}$, $e(1) = \{1\}$ and $e(x) = \{0, 1\}$.

Next, we identify in which gadgets the random values are being used, and we also consider the cases where a random value leaks into a gadget. We can then introduce three conditions that violate our composition policy. First, a random used for an input of a gadget execution should not leak in another PG gadget, or in a $\overline{\text{PG}}$ gadget that leaks in transition with a PG gadget. The second case is the joint leakage of a random is used for

$$\begin{array}{c}
\text{RNDRND} \quad \frac{x \in \mathcal{I}_C \quad \kappa(x, t) = \text{rnd}}{x[t] : \text{Rnd}((x, t))} \quad \text{RNDREG} \quad \frac{y \leftarrow \text{Reg}(x) \in \mathcal{A}_C \quad x[t] : \text{Rnd}(r)}{y[t+1] : \text{Rnd}(r)} \quad \text{GATEBIJ} \quad \frac{\forall (x_1, \dots, x_m) \in e(v_1) \times \dots \times e(v_m), \quad \overline{f_F}(x_1, \dots, x_{i-1}, x, x_{i+1}, \dots, x_m) = x}{F : \text{Bij}(i, (v_1, \dots, v_m))} \\
\\
\text{RNDCOMB} \quad \frac{\begin{array}{l} y \leftarrow F(x_1, \dots, x_m) \in \mathcal{A}_{\mathcal{E}_C} \quad F \in \mathcal{F}_{\text{comb}} \\ x_i[t] : \text{Rnd}(r) \quad \forall j \in \llbracket m \rrbracket : j \neq i \Rightarrow \neg(x_j[t] : \text{Rnd}(r)) \\ F : \text{Bij}(i, (V(x_1[t]), \dots, V(x_m[t]))) \end{array}}{y[t] : \text{Rnd}(r)} \quad \text{RNDUSE} \quad \frac{\begin{array}{l} A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \\ A \in \mathcal{A}_C \quad F \in \mathcal{F}_{\text{gadget}} \\ A : \text{PG}(t) \quad \kappa_F^I(i) = \text{rnd} \quad x_i[t + \ell_F^I(i)] : \text{Rnd}(r) \end{array}}{r : \text{RU}((A, t, i))} \\
\\
\text{RNDGADGETLEAK} \quad \frac{\begin{array}{l} A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \\ A \in \mathcal{A}_C \quad F \in \mathcal{F}_{\text{gadget}} \quad \text{rnd}_r \in \text{GD}(x_i[t + \ell_F^I(i)]) \end{array}}{r : \text{RL}((A, t, i))} \quad \text{RNDUSEUNIQUE} \quad \frac{\begin{array}{l} r : \text{RU}(u) \quad r : \text{RL}((A, t, i)) \\ u \neq (A, t, i) \quad A : \text{PG}(t') \quad t' \in \{t-1, t, t+1\} \end{array}}{(C, \kappa) : \text{PINI}} \\
\\
\text{RNDLEAKSHARE} \quad \frac{\begin{array}{l} \text{GTE}(S) \quad \text{rnd}_r, \text{sh}_i \in S \quad r : \text{RU}(u) \end{array}}{(C, \kappa) : \text{PINI}} \quad \text{RNDLEAKRND} \quad \frac{\begin{array}{l} \text{GTE}(S) \quad \text{rnd}_r, \text{rnd}_{r'} \in S \quad r \neq r' \quad r : \text{RU}((A, t, i)) \\ r' : \text{RU}((A', t', i')) \quad (A', t', i') \notin \{(A, t-1, i), (A, t+1, i)\} \end{array}}{(C, \kappa) : \text{PINI}}
\end{array}$$

Figure 5: Analysis of randomness usage and leakage.

$$\begin{array}{c}
\text{GADGETINPUTSHARE} \quad \frac{\begin{array}{l} (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \in \mathcal{A}_C \\ F \in \mathcal{F}_{\text{gadget}} \quad \kappa_F^I(j) = \text{sh}_i \end{array}}{x_j[t + \ell_F^I(j)] : \text{IsShare}(i)} \quad \text{CIRCUITOUTPUTSHARE} \quad \frac{x \in \mathcal{O}_C \quad \kappa(x, t) = \text{sh}_i}{x[t] : \text{IsShare}(i)} \quad \text{FAILWRONGSHARE} \quad \frac{x[t] : \text{IsShare}(i) \quad \text{sh}_{i'} \in \text{GD}(x[t]) \quad i \neq i'}{(C, \kappa) : \text{PINI}} \\
\\
\text{FAILGADGETRANDOM} \quad \frac{\begin{array}{l} A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m) \\ A : \text{PG}(t) \quad \text{rnd} = \kappa_F^I(i) \quad \neg(x_j[t + \ell_F^I(j)] : \text{Rnd}(r)) \end{array}}{(C, \kappa) : \text{PINI}} \quad \text{FAILOUTPUTRANDOM} \quad \frac{x \in \mathcal{O}_C \quad \text{rnd}_r \in \text{GD}(x[t]) \quad r : \text{RU}(u)}{(C, \kappa) : \text{PINI}}
\end{array}$$

Figure 6: Analysis of gadget inputs and circuit outputs.

a gadget and of a share (outside the gadget). The third violation is the presence joint leakage of two randoms used in gadgets (except if these randoms are necessarily leaking in transition by being on one input wire of the same gadget for two consecutive clock cycles), happening in a simple gate or in a PG gadget.

Secure gadget execution and output shares Let us finally turn to the gadget inputs: input shares can only be sensitive in the correct share index, while randomness input of PG gadgets must be fresh randoms. Also, output shares of the circuit can only be sensitive in the correct share index, and they should not leak used randomness.

4.2 Security proof

Theorem 1. *Let (C, κ) be an extended circuit in which all the gadget gates correspond to d -PINI or d -OPINI gadgets and the gadget gates are well-formed gates¹⁰. If the inference rules of Section 4.1 cannot conclude $(C, \kappa) : \text{PINI}$, then for any $T \in \mathbb{N}$, (C, κ) is d -PINI for T clock cycles.*

Before giving the full-length proof of Theorem 1, we provide a sketch that explains the main ideas informally. The proof is structured in three parts.

In the first part, we show that we can order all the executions of assignments (i.e., order $\mathcal{A}_C \times \mathbb{N}$) in such a way that the evaluation of all gate executions can be done in this

¹⁰The restriction of gadget gates to well-formed gates means that gadgets do not take any input after producing any of their outputs. This assumption is not strictly necessary for security, but it simplifies the proof. This simplifying assumption is not limiting: all practical elementary PINI and OPINI gadgets are well-formed.

order, with the values of wires computed before they are used. Further, as a preliminary to proving the security for transitions between executions of PINI gadgets, we partition this set of executions into clusters where any set of consecutive executions of an assignment with a PINI gadget is a single cluster, and all other executions have their own cluster of size 1. We show that the clusters are also ordered in such a way that all inputs of all assignments in each cluster are computed before evaluating the assignments in the cluster. Looking at the probes, we virtually move each probe on a wire that carries a random bit to the input wire of the gadget that uses it: this will more accurately represent the adversary's knowledge when we apply the (O)PINI definitions to the gadgets. As a conclusion to the first part of the proof, the probes are partitioned according to the cluster of their source assignment.

The second part of the proof computes the set of input shares I in the PINI definition. Starting from the end of the execution and the set of probed output share indices O , we go backwards through the list of clusters. For each cluster C_i , we get a new set of probed share indices O_{i-1} . If a cluster contains executions of a PINI gadget, we get the new set of share indices by applying the PINI property of that gadget with the current set of shares indices O_i as output share indices and the union of all probes in the cluster as the set of probes. If a cluster contains the execution of an assignment A with an OPINI gadget gate, we perform a similar operation, but we take the set of probes across all executions of the assignment, and we remove from the set of output share indices all the share indices that have been added in O_i due to clusters containing executions of the assignment A . Finally, if a cluster contains a combinational gate or a register whose output is glitch-dependent on some share index, that share index is added to the set of indices. By construction, the size of share indices satisfies the constraint of the PINI definition.

The third and last part proves the equality of the distributions in the PINI definition. We show that for any cluster C_i , we can simulate all wires in clusters before C_i that are glitch-dependent on share indices in O_{i-1} , and all wires that are not glitch-dependent and do not depend on a used random. Further, we can simulate the probes in gadgets (using the simulators of the gadgets). Finally, we show that we can simulate the probes wires that glitch-depend on used randomness, thanks to the simulation of the randomness by the simulator of the gadget that uses the randomness.

Proof of Theorem 1. Before detailing the three parts of the proof, we introduce some common definitions. $T' = \max(T, \max_{(x,t) \in P} t)$ is the last clock cycle we care about. We then consider the wires in the executions: $\mathcal{W}_C \times \llbracket T' \rrbracket$, and we denote by W_r^{det} the subset of wires that depend on a used random and by W_r^{glitch} the subset of wires that depend on a used random: W_r^{det} (respectively, W_r^{glitch}) is the set of $(x, t) \in \mathcal{W}_C \times \llbracket T' \rrbracket$ such that there exists r, u satisfying $\text{rnd}_r \in D(x[t])$ (respectively $\text{rnd}_r \in \text{GD}(x[t])$) and $r : \text{RU}(u)$. Further, for any $(x, t) \in W_r^{\text{det}}$ or $(x, t) \in W_r^{\text{glitch}}$, we denote by $U((x, t))$ the associated u . We also consider the dependency on share indices: for all $i \in \mathbb{N}$, $W_{\text{sh}_i}^{\text{det}} = \{(x, t) \in \mathcal{W}_C \times \llbracket T' \rrbracket \mid i \in D(x[t])\}$ and $W_{\text{sh}_i}^{\text{glitch}} = \{(x, t) \in \mathcal{W}_C \times \llbracket T' \rrbracket \mid i \in \text{GD}(x[t])\}$. For any $X \subset \mathbb{N}$, we also define $W_{\text{sh}(X)}^{\text{det}} = \bigcup_{i \in X} W_{\text{sh}_i}^{\text{det}}$ and $W_{\text{sh}(X)}^{\text{glitch}} = \bigcup_{i \in X} W_{\text{sh}_i}^{\text{glitch}}$.

Let us now detail the three parts of the proof, as introduced in the sketch.

(First part: we build clusters.) Let $\prec_w$ be a homogeneous binary relation over the set $\mathcal{A}_C \times \mathbb{N}$ where $(A_1, t_1) \prec_w (A_2, t_2)$ if and only if there exists $x \in \mathcal{W}_C$ which is the j th result of A_1 and x is the i th operand of $A' \in \mathcal{A}_C$, and there exists $t \in \mathbb{N}$ such that $t_1 = t + \ell_{F_2}^I(i)$ and $t_2 = t + \ell_{F_1}^O(j)$, where F_1 (respectively F_2) is the gate in A_1 (resp. A_2). Let us remark that $\prec_w$ is irreflexive: if $A \prec_w A$, then the corresponding witness wire x and A form a trivial combinational loop, contradicting the well-formedness of $\mathcal{C}$. Let $\prec$ be the transitive closure of $\prec_w$, we will prove that $\prec$ is a strict partial order. Since $\prec$ is transitive by definition, it remains to prove that it is irreflexive and asymmetric. First, for asymmetry, if $(A, t) \prec (A', t')$ and $(A', t') \prec (A, t)$, then there exists $((A_i, t_i))_{i \in \llbracket k \rrbracket}$ such that

$(A_1, t_1) = (A_k, t_k) = (A, t)$ and for all $i \in \llbracket k-1 \rrbracket$, $(A_i, t_i) \prec_w (A_{i+1}, t_{i+1})$. Let F_i be the gate in the assignment A_i , we have $t_k = t_1 + \ell_{F_1}^O(j_1^O) + \left(\sum_{i=2}^{k-1} \ell_{F_i}^O(j_i^O) - \ell_{F_i}^I(j_i^I) \right) - \ell_{F_k}^I(j_k^I)$ for some indices j_i^I and j_i^O . With the equalities $F_1 = F_k$ and $t_k = t_1$, we can rewrite the previous equation as $\sum_{i=1}^{k-1} \ell_{F_i}^O(j_i^O) - \ell_{F_i}^I(j_i^I) = 0$ where we define $j_1^I = j_k^I$. Since all the gates are well-formed gates, $\ell_{F_i}^O(j_i^O) \geq \ell_{F_i}^I(j_i^I)$, hence the equation implies $\ell_{F_i}^O(j_i^O) = \ell_{F_i}^I(j_i^I)$ for all $i \in \llbracket k-1 \rrbracket$, which means that we contradict the well-formedness of $\mathcal{C}$ by exhibiting a combinational loop. Therefore, $\prec$ is asymmetric. Finally, since $\prec_w$ is irreflexive, $(A, t) \prec (A, t)$ implies the existence of (A', t') such that $(A, t) \prec (A', t')$ and $(A', t') \prec (A, t)$ which contradicts the asymmetry of $\prec$, proving the $\prec$ is irreflexive.

Let $T' = \max(T, \max_{(x,t) \in P} t)$ be the last cycle we care about. Since $\mathcal{A}_C \times \llbracket T' \rrbracket$ is a finite partially ordered set, it can be enumerated as $\mathcal{A}_C \times \llbracket T' \rrbracket = \{(A_1, t_1), \dots, (A_k, t_k)\}$ such that for all $i \in \llbracket k-1 \rrbracket$, $(A_i, t_i) \prec (A_{i+1}, t_{i+1})$, forming a total order. Moreover, we can choose this order such that if the gate of the assignment A belongs to $\mathcal{F}_{\text{PINI}}$ and for some $t' < t''$, $A : \text{PG}(t)$ for all t such that $t \in \llbracket t', t'' \rrbracket$, then $((A, t'), \dots, (A, t''))$ is a substring of $((A_1, t_1), \dots, (A_k, t_k))$: the DPGEPINIGADGET , DPGEASSIGN and FAILDPGE rules imply that there is no order in $((A, t'), \dots, (A, t''))$ with respect to $\prec$. Following this observation, we partition $\mathcal{A}_C \times \llbracket T' \rrbracket$ into *clusters*, let us denote by C^* the set of all clusters. The partition is built as follows: for any $A \in \mathcal{A}_C$ whose gate belongs to $\mathcal{F}_{\text{PINI}}$, if $\llbracket t, t' \rrbracket$ is a maximal (i.e., it cannot be extended from either end) such that $A : \text{PG}(t'')$ for all $t'' \in \llbracket t, t' \rrbracket$, then $\{A\} \times \llbracket t, t' \rrbracket \in C^*$. For any $(A, t) \in \mathcal{A}_C \times \llbracket T' \rrbracket$ not covered by the previous rule, we let $\{(A, t)\} \in C^*$. It then follows that $C^* = \{C_1, \dots, C_{k'}\}$ with $k' = |C^*|$, such that for any $i, i' \in \llbracket k' \rrbracket$ such that $i < i'$ and any $(A_j, t_j) \in C_i$, $(A_{j'}, t_{j'}) \in C_{i'}$, it holds that $j < j'$.

Let us now consider the probes. Informally, there are three kinds of probes: probes within a gadget, probes outside gadgets that depend on a random value used in a PG gadget, and the remaining probes (probes outside gadgets that may depend on a share index). To formalize this partition, we define $P_r = P \cap W_r^{\text{glitch}}$. Further, for any $p \in P_r$, we pose $(A, t, i) = U(p)$ and denote by $\mathcal{G} = (\mathcal{C}_G, \kappa_G)$ the gadget corresponding to the gate of A . Let $x \in \mathcal{W}_{\mathcal{C}_G}$ be the wire associated to the i th input of the gate, we define $\mu(p) = \mathcal{E}_{\mathcal{C}_G, t}^{A, x}$. We can now build the set of probes. This gives us our set of probes with the probes of P_r moved to the gadget inputs: $P' = (P \setminus P_r) \cup \{\mu(p) : p \in P_r\}$. Let us now partition P' into sets $P'(A, t)$ for $A \in \mathcal{A}_C$, $t \in \llbracket T' \rrbracket$. For any $p \in P'$, if there exists $A \in \mathcal{A}_C$, $t \in \llbracket T' \rrbracket$ and $x \in \mathcal{W}_{\mathcal{C}_G}$ such that $\mathcal{E}_{\mathcal{C}_G, t}^{A, x} = p$ where $(\mathcal{C}_G, \kappa_G)$ is the gadget associated to the gate of A , then we put p in $P'(A, t)$ (if there are multiple such A, t, x , then one of them is arbitrarily chosen). If such A, t, x do not exist, then we put $p = (x, t)$ in $P'(\text{src}(x), t)$ if $\text{src}_{\mathcal{C}}(x)$ exists, while otherwise, x is input wire which is an operand of an assignment with a simple gate (combinational or Reg) and we may safely ignore this case: the values observed by p are a subset of the values observed by a probe on the result of that assignment. Finally, we map the probes to clusters: for any cluster C_i , we pose $C_i^{P'} = \bigcup_{(A, t) \in C_i} P'(A, t)$, hence $\{C_i^{P'} : i \in \llbracket k' \rrbracket\}$ is a partition of P' .

(Second part: we build I , the set of input share indices.) In this part, we build a sequence $(I_i)_{i \in \llbracket k' \rrbracket}$ of sets of share indices by induction on i from k' down to 1, and $I = I' \cup \bigcup_{i=1}^{k'} I_i$, where I' will be defined later. In parallel, we have $O_{i-1} = O_i \cup I_i$ with $O_{k'} = O \cup I'$ as the base case. Informally, the set O_i is the set of share indices needed to simulate all probes in $C_j^{P'}$ for $j > i$.

For this induction, we also compute $I_{\text{OPINI}, i} : \mathcal{A}_C \rightarrow \mathcal{P}(\mathbb{N})$, which, for any assignment with an OPINI gadget gate, represents the set of share indices contributed by this assignment to O_i across all executions of that gadget in $C_{i+1}, C_{i+2}, \dots$. As the base case, $I_{\text{OPINI}, k'}(A) = \emptyset$ for all $A \in \mathcal{A}_C$.

Let us now define I' . Informally, I' is the set of share indices needed to simulate the probes outside gadgets. Let $\Omega \subset \llbracket k' \rrbracket$ be the set indices of the clusters that contain a simple

gate assignment (i.e., the cluster is $\{(A, t)\}$ where A 's gate belongs to $\mathcal{F}_{\text{comb}} \cup \{\text{Reg}\}$). For each $i \in \Omega$, let $\{(A, t)\} = C_i$. If $C_i^{P'} = \emptyset$, then we pose $I'_i = \emptyset$. Otherwise, $|C_i^{P'}| = 1$ since A has only one result, and we pose $C_i^{P'} = \{(x, t')\}$. If there exists $j \in \mathbb{N}$ such that $W_{\text{sh}_j}^{\text{glitch}} \cap \{(x, t'), (x, t' + 1)\} \neq \emptyset$, we pose $I'_i = \{j\}$, otherwise $I'_i = \emptyset$. Overall, we get $I' = \bigcup_{i \in \Omega} I'_i$, and we let $I'_i = \emptyset$ for all $i \notin \Omega$.

Let us now formally describe the induction step for I_i . We distinguish three cases depending on C_i .

In the first case, $C_i = \{(A, t) : t \in \llbracket t', t'' \rrbracket\}$ for some $A \in \mathcal{A}_{\mathcal{C}}$ whose gate F belongs to $\mathcal{F}_{\text{PINI}}$. Let $\mathcal{G} = (\mathcal{C}_{\mathcal{G}}, \kappa_{\mathcal{G}})$ be the gadget corresponding to F , we map the probes in $C_i^{P'}$ to $\mathcal{W}_{\mathcal{C}_{\mathcal{G}}}$: $\mathcal{W}_i^{P'} = \{S_{\mathcal{C}}^W(x) : (x, t) \in C_i^{P'}\}$. We then apply the gadget PINI definition for $\mathcal{G}$ with $\mathcal{W}_i^{P'}$ as the set of probes and O_i as the set of output share indices, getting I_i as the set of input share indices. We let $I_{\text{OPINI}, i-1} = I_{\text{OPINI}, i}$.

In the second case, $C_i = \{(A, t)\}$ where A 's gate belongs to $\mathcal{F}_{\text{OPINI}}$. Let $\mathcal{G} = (\mathcal{C}_{\mathcal{G}}, \kappa_{\mathcal{G}})$ be the corresponding OPINI gadget and let $\mathcal{W}_i^{P'} = \{S_{\mathcal{C}}^W(x) : (x, t) \in \bigcup_{\{j: \exists t', C_j = (A, t')\}} C_j^{P'}\}$ be the set of all probes in the executions of the assignment A , mapped to $\mathcal{W}_{\mathcal{C}_{\mathcal{G}}}$. We apply the gadget OPINI definition for $\mathcal{G}$ with $\mathcal{W}_i^{P'}$ as the set of probes and $O_i \setminus I_{\text{OPINI}, i}$ as the set of output shares indices, getting I_i as the set of input share indices. Then, we pose $O_{\text{OPINI}, i-1} = I_i$.

The last case is $C_i = \{(A, t)\}$ where A 's gate belongs to $\mathcal{F}_{\text{comb}} \cup \{\text{Reg}\}$. This case has already been covered, hence we set $I_i = \emptyset$ and let $I_{\text{OPINI}, i-1} = I_{\text{OPINI}, i}$.

Let us now bound $|I|$. For any i such that C_i corresponds to an assignment with a simple gate or to assignments with a PINI gadget gate, we have $|I_i \cup I'_i| \leq |C_i^{P'}|$ by construction, and we denote by X the set of such i . For each assignment $A \in \mathcal{A}_{\mathcal{C}}$ whose gate is an OPINI gadget gate, let Y_A be the set of all i such that the assignment in C_i is A . Then, by construction, $|I_i| \leq \sum_{i' \in Y_A} |C_{i'}^{P'}|$ for all $i' \in I_i$. Further, assuming wlog that the set of input share indices in the OPINI definition is monotonic with respect to the set of output share indices (e.g., this holds if the input set is always chosen to be minimal), let $i^* = \arg \max_{i \in Y_A} |I_i|$, we have $I_i \subset I_{i^*}$ for all $i \in Y_A$, and therefore $|\bigcup_{i \in Y_A} I_i| \leq \sum_{i' \in Y_A} |C_{i'}^{P'}|$. Overall, taking the union over X and over all assignments A with an OPINI gadget gate, we find $|I| = \left| \bigcup_{i \in \llbracket k' \rrbracket} I_i \cup I'_i \right| \leq \sum_{i \in \llbracket k' \rrbracket} |C_i^{P'}| = |P'| = |P| = d_1$, hence I satisfies the size constraint of the PINI definition.

(Third part: we show how to simulate the probes and the outputs.) Let us partition the wires following the cluster where they are computed: for any $i \in \llbracket k' \rrbracket$, $C_i^W = \{(x, t) \in \mathcal{W}_{\mathcal{C}} \times \llbracket T' \rrbracket \mid (\text{src}_{\mathcal{C}}(x), t) \in C_i\}$. We also have a cluster for input wires: $C_0^W = \mathcal{I}_{\mathcal{C}} \times \llbracket T' \rrbracket$.

Let $C_i^{W^{\text{det}}}$ be the set of wires in C_i^W except those that depend on a used random or depend on a share that does not belong to O_i : $C_i^{W^{\text{det}}} = C_i^W \setminus (W_r^{\text{det}} \cup W_{\text{sh}(\mathbb{N} \setminus O_i)}^{\text{det}})$. Similarly, we look at glitch-dependencies: $C_i^{W^{\text{glitch}}} = C_i^W \setminus (W_r^{\text{glitch}} \cup W_{\text{sh}(\mathbb{N} \setminus O_i)}^{\text{glitch}})$. We also define $C_{<i}^{W^{\text{det}}} = \bigcup_{j=0}^{i-1} C_j^{W^{\text{det}}}$, and $C^{W^{\text{det}}} = C_{<k'+1}^{W^{\text{det}}}$, along with similar definitions for $C_{<i}^{W^{\text{glitch}}}$ and $C^{W^{\text{glitch}}}$.

As a preliminary remark, let us show that for any $(x, t) \in \mathcal{W}_{\mathcal{C}} \times \llbracket T' \rrbracket$ and $i \in O_0$ for which $\text{sh}_i \in \text{GD}(x[t])$, $(x, t) \in C^{W^{\text{glitch}}}$. Indeed, there exists $j \in \llbracket k' \rrbracket$ such that $(x, t) \in C_j^W$ by construction of the clusters. By `GLITCHTRANSLEAK` and `FAILDEPSHARES`, there is no $\text{sh}_{i'}$ in $\text{GD}(x[t])$ with $i' \neq i$. Further, by `RNDLEAKSHARE`, there is no rnd_r in $\text{GD}(x[t])$ with $r : \text{RUu}$. Therefore, (x, t) belongs to $C_j^{W^{\text{glitch}}}$. The same property holds do non-glitch dependencies: if $\text{sh}_i \in \text{D}(x[t])$, then $(x, t) \in C^{W^{\text{det}}}$. Indeed, $\text{sh}_i \in \text{D}(x[t])$ implies $\text{sh}_i \in \text{GD}(x[t])$, and by construction $C_i^{W^{\text{glitch}}} \subset C_i^{W^{\text{det}}}$.

Given a compatible glitchy input valuation f , let g be the minimal $(I \cup O)$, f -equivalent glitchy input valuation. Without using f , we build a simulator $S^{g, \rho_{\mathcal{R}}, \text{det}}$ such that $S^{g, \rho_{\mathcal{R}}, \text{det}}(x, t) = (S^{g, \rho_{\mathcal{R}}, \text{det}}(x, t), S^{g, \rho_{\mathcal{R}}, \text{glitch}}(x, t))$ simulates jointly the value of the wires in $C^{W^{\text{det}}}$ and glitches on the wires in $C^{W^{\text{glitch}}}$ in an execution with input f , where $\rho_{\mathcal{R}}$ is associated to the f -uniform set of valuations $\mathcal{R}$ in the simulation definition.

We proceed by induction on $i = 0, \dots, k'$. For the base case, we have $O_0 = I \cup O$. For all wires $(x, t) \in \mathcal{I}_{\mathcal{C}} \times \llbracket T' \rrbracket$ such that $\kappa(x, t) \in \mathcal{K}_p \cup \text{sh}(O_0)$, the simulation is trivial: $S^{g, \rho_{\mathcal{R}}, \text{det}}(x, t) = g(x, t)$. If $\kappa(x, t) = \text{rnd}$ and $(x, t) \notin W_r$, we sample a uniformly (α, β) from $\rho_{\mathcal{R}}(x, t)$ and let $S_0^g = (\alpha, \beta)$.

Next, assume by induction that $S^{g, \rho_{\mathcal{R}}, \text{det}}(x, t)$ (resp. $S^{g, \rho_{\mathcal{R}}, \text{glitch}}(x, t)$) is already well-defined for all $(x, t) \in C_{<i}^{W^{\text{det}}}$ (resp. $(x, t) \in C_{<i}^{W^{\text{glitch}}}$). We show how to extend $S^{g, \rho_{\mathcal{R}}, \text{det}}$ (resp. $S^{g, \rho_{\mathcal{R}}, \text{glitch}}$) to additionally simulate the observations on the wires $C_i^{W^{\text{det}}}$ (resp. $C_i^{W^{\text{glitch}}}$) in a way that is consistent with respect to the other simulated values. There are three cases for C_i .

In the first case, C_i contains consecutive executions of an assignment A whose gate is a PINI gadget gate for a gadget $\mathcal{G}$, for some interval $\llbracket t', t'' \rrbracket$. Since C_i^W contains the output shares of these executions, $C_i^{W^{\text{det}}} = C_i^{W^{\text{glitch}}}$ and the share index of any wire in $C_i^{W^{\text{det}}}$ belongs to O_i , by DEPPGADGET. We can simulate the value of these wires as well as their glitches using the PINI simulator for $\mathcal{G}$. This simulator needs knowledge of its input shares with index in O_{i-1} , which are wires (x, t) such that for all $\text{sh}_j \in \text{GD}(x[t])$, $j \in O_{i-1}$ (thanks to GADGETINPUTSHARE and FAILWRONGSHARE). By construction of the clusters, we also know that for any such (x, t) , there exists $i' < i$ with $(x, t) \in C_{i'}^W$. If there exists $j \in \mathbb{N}$ such that $\text{sh}_j \in \text{GD}(x[t])$, then $(x, t) \in C_{i'}^{W^{\text{glitch}}}$ by the preliminary remark. Otherwise, by RNDGADGETLEAK and RNDUSEUNIQUE, we know that there does not exist any r, u satisfying $\text{rnd}_r \in \text{GD}(x[t])$ and $r : \text{RU}(u)$, hence $(x, t) \in C_{i'}^{W^{\text{glitch}}}$. Therefore, $(x, t) \in C_{i'}^{W^{\text{glitch}}} \subset C_{i'}^{W^{\text{det}}}$ and we have already simulated the values of the required input shares (values and glitches) for the PINI gadget simulator (for executions in the interval $\llbracket t', t'' \rrbracket$). We also need to show that the input randomness of the PINI gadget executions are uniformly random and have not been simulated yet. Any input randomness of the gadget is in bijection with an input random of the circuit by RNDRND, RNDREG, GATEBIJ, RNDCOMB and FAILGADGETRANDOM. Further, by RNDUSE, this input random does not belong to W_r and has therefore not been simulated in the base case of the induction, while RNDUSEUNIQUE ensures that it has not been simulated by another (O)PINI gadget simulator. This concludes the proof that we can correctly run the PINI gadget simulator for all executions in $\llbracket t', t'' \rrbracket$. Running the simulator gives us glitch-extended probes on the outputs of the gadget execution whose share index belongs to O_i . By DEPPGADGET, this corresponds exactly to the wires in $C_i^{W^{\text{det}}} = C_i^{W^{\text{glitch}}}$.

In the second case, $C_i = \{(A, t)\}$ where A 's gate is an OPINI gadget gate. Similarly to the first case, we can use the OPINI gadget simulator to simulate probes on the outputs with share index in O_i , if we can provide it with the values of glitch-extended probes on its input shares with index in O_{i-1} and guarantee that the input randomness is uniform and not simulated elsewhere. This can be proven with exactly the same arguments as in the first case (PINI gadgets).

In the third case, $C_i = \{(A, t)\}$ where A 's gate is a simple gate, and we denote by x its output, and (x, t) is the only wire we have to consider. If the gate is a combinational gate, then $(x, t) \in C_i^{W^{\text{glitch}}}$ implies that each operand wire belongs to some $C_j^{W^{\text{glitch}}}$ for $j < i$, by GLCOMB, GLITCHDEPCOMB and the ordering of clusters. Simulating a glitch-extended probe on (x, t) then amounts to concatenating the already-simulated glitch-extended probes on the operands. For $S^{g, \rho_{\mathcal{R}}, \text{det}}(x, t)$, if $(x, t) \in C_i^{W^{\text{det}}}$, we can simulate it trivially if $V(x[t]) \in \{0, 1\}$. Otherwise, by DEPCOMBX and the ordering of clusters, we know that each operand wire belongs to some $C_j^{W^{\text{det}}}$ for $j < i$, hence we can simply compute the

gate operation. If the gate is **Reg**, $C_i^{W^{\text{det}}} = C_i^{W^{\text{glitch}}}$ by **DEPREG**, and if $(x, t) \in C_i^{W^{\text{det}}}$, the operand belongs to $C_j^{W^{\text{det}}}$ for some $j < i$. The output of the register (which does not carry glitches) can thus be simulated from the value of the operand.

We now have the simulation of glitch-extended probes for all wires in $C^{W^{\text{glitch}}}$ and simulation of the values of the wires in $C^{W^{\text{det}}}$. Actually, this simulation also includes glitch-extended probes on all wires in P' . Indeed, the PINI and OPINI simulator take care of simulating them for the first and second cases of clusters. For the third case with a cluster C_i , the only wire to consider is the output wire (x, t) of the gate, and we assume it belongs to P' . If there exists $j \in \mathbb{N}$ such that $\text{sh}_j \in \text{GD}(x[t])$, then $j \in I'_i$, hence all the operands belong to $C_{<i}^{W^{\text{glitch}}}$ (and to $C_{<i}^{W^{\text{det}}}$). They are therefore simulated, making the simulation of a glitch-extended probe on $x[t]$ trivial. Otherwise, since (x, t) cannot be a used random (it would have been mapped to the inside of a gadget in P' by μ), it must belong to $C_i^{W^{\text{glitch}}}$ and a glitch-extended probe on it has therefore been simulated.

Let us now extend the simulation of the probes with the transitions, we only need to cover the probes in P' and not the output shares.

First let us consider a cluster C_i that contains consecutive executions for a time interval $\llbracket t', t'' \rrbracket$ of an assignment A whose gate is a PINI gadget gate. The transition between two executions of the interval are trivial since the simulation of each gadget execution simulates probes on the wires corresponding to all probes placed on the executions of the cluster. For the execution after the last one of the interval (i.e., the execution at $t'' + 1$), we know that the input wires are not glitch-dependent on any share by **NPGadget** nor glitch-dependent on any used randomness by **RndUseUnique**, hence those wires belong to $C^{W^{\text{glitch}}}$, meaning that glitch-extended probes on them along with their value have already been simulated and can be used to simulate any glitch-extended probe in the execution of the gadget.

For the second case, $C_i = \{(A, t)\}$ where A 's gate is a OPINI gadget gate. Since all the simulation of A use the same set of probes for the OPINI simulator, the simulation of the transitions is trivial.

For the third case, $C_i = \{(A, t)\}$ where A 's gate is a simple gate. Let i' be such that $C_{i'} = \{(A, t + 1)\}$, and let x be the output wire of A . Since the construction of I'_i takes into account $(x, t + 1)$ in the same manner as (x, t) and $I'_i \subset O_{i'}$, the argument for the simulation of the probe (x, t) also applies to $(x, t + 1)$.

Finally, we get back to P : it remains to simulate the probes in P_r . By **GATEBIJ**, **RNDCOMB**, we know that for any $(x, t) : \text{RU}(u)$, there is a bijection b mapping the value of $x[t]$ to the value of $x'[t']$, where $x'[t']$ is the wire corresponding to u (input of the gadget). Further, this bijection cannot depend on any share by **RNDLEAKSHARE** nor any other used random by **RNDLEAKRND**. Therefore, the bijection only depends on inputs that have already been simulated, and we can get a simulation $S^{g, \rho_{\mathcal{R}}, \text{det}}(x, t)$ by applying the inverse of b to the value of $x'[t']$ provided by the corresponding gadget simulator. We then sample (α, β) uniformly from $\rho_{\mathcal{R}}^{S^{g, \rho_{\mathcal{R}}, \text{det}}(x, t)}(x, t)$ in order to simulate the glitches on $x[t]$ as $S^{g, \rho_{\mathcal{R}}, \text{glitch}}(x, t) = \beta$. Once all the input randoms are simulated, the values and extended probes of wires P_r can be simulated (they do not depend on any share, by **RNDLEAKSHARE**). Finally, we update the simulation of the glitches inside gadget: any input random of the gadget should be replaced by the glitches on that value (again, since this is used randomness, there is no dependency on shares in the glitches). $\square$

5 Verification through symbolic execution

In this section, we describe how to turn the rules of [Section 4.1](#) into an efficient security verification tool. While a simple approach would be to encode all the rules in a logic programming language such as Prolog, for a bounded time interval $\llbracket 0, T \rrbracket$. However, proving security requires analyzing a very large set of predicates to rule out any possible failure rule.

Indeed, we have to cover every assignment $\mathcal{A}_C$ for all clock cycles in $\llbracket 0, T \rrbracket$. For each of those, rules such as the ones for DPGE and random usage/leakage also potentially interact with all other assignments. Therefore, the complexity may rise to $\mathcal{O}(|\mathcal{A}_C|^2 T^2)$. With this complexity, even masked circuit of moderate size: for example, a small masked AES implementation is already challenging to verify. Indeed, those easily reach $|\mathcal{A}_C| = 1000$ (as a starting point, a $d + 1$ -shares AES-128 needs $256(d + 1)$ Reg gates for the state/key storage only), and may need more than $T = 100$ clock cycles [MCS22, CGM⁺24].

We therefore use a more specialized approach which is based on symbolic simulation. This simulation tracks properties of each wire execution, such as V, D and GD. It also uses global tables when necessary, for example to verify usage unicity of each input random. This approach allows the verification's memory use to be independent of T , since by performing the security check on-the-fly, we do not need the whole simulation history. Let us now describe in more details the components of the symbolic simulation.

Basic simulation algorithm The simulation iterates over clock cycles $t \in \llbracket 0, T \rrbracket$. Since $\mathcal{E}_C$ is a well-formed simple circuit, we can enumerate its assignments in a way that preserves combinational ordering. We can thus compute $V((x, t))$ for all wires $x \in \mathcal{E}_C$ using this order and the valuation of the input wires of the Regs at the previous clock cycle. For other properties, we use $\mathcal{C}$. Since it does not contain combinational loops, we can enumerate assignments in $\mathcal{A}_C$ in a way that respects combinational dependencies. Then, when running the simulation for clock cycle t , for each assignment $A \equiv (y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_m)$ we compute properties the output wires of $(A, t - \max_{i \in \llbracket m \rrbracket} \ell_F^I(i))$. That is, we compute the outputs of a gate or gadget as soon as all its input wires have been simulated. This requires storing the properties of the input wires of each gadget across $\max_{i \in \llbracket m \rrbracket} \ell_F^I(i)$ clock cycles, and the properties of the output wires for $\max_{i \in \llbracket n \rrbracket} \ell_F^O(i) - \max_{i \in \llbracket m \rrbracket} \ell_F^I(i)$ clock cycles. When evaluating a gadget execution (A, t) , we also check whether $A : \text{PG}(t)$.

Concretely, for each $x[t]$, the two basic properties we store are $D(x[t])$ and $GD(x[t])$ (along with $V(x[t])$, whose computation is already explained). Computing D and GD simply follows the rules of Section 4.1, analyzing the wires inside the gadget if $A : \overline{\text{PG}}(t)$. When computing these wire properties, we also check FAILDEPSHARES and FAILWRONGSHARE.

Transitions between PINI gadget executions For each wire (x, t) , we further keep a map $\mathcal{A}_C \rightarrow \mathbb{N}$, which, for any assignment A whose gate belongs to $\mathcal{F}_{\text{PINI}}$, stores $\max\{t' \in \mathbb{N} \mid x[t] : \text{DPGE}(t', A)\}$. For any assignment, the map for the output wires is the element-wise maximum of the maps of the operand wires, updated with the current assignment's latency if the assignment's gate belongs to $\mathcal{F}_{\text{PINI}}$. Further, since many wires (e.g., all output shares of a gadget execution) share the same map, using an interning datastructure for storing the maps saves a significant amount of memory. Then, when evaluating the execution of an assignment A with a PINI gadget, we use the maps of the input wires to check FAILDPGE. For this verification, we also need to know the largest t' such that $t' < t$ and $A : \text{PG}(t')$. This information can be obtained by storing the clock cycle of the last PG execution for every PINI gadget (and updating it when the gadget execution is evaluated).

Randomness For the analysis of randomness, let us first remark that in a circuit that passes verification, we cannot have $x[t] : \text{Rnd}(r)$ and $x[t] : \text{Rnd}(r')$ with $r \neq r'$ for any wire $x[t]$ which is a randomness input of a PG gadget. Indeed, in such a case, we have $r : \text{RU}(u)$ and $r' : \text{RL}(u)$, thus RNDUSEUNIQUE concludes $\overline{\text{PINI}}$. Further, if the wire $x[t]$ is not a randomness input of a PG gadget, the properties $x[t] : \text{Rnd}(r)$ and $x[t] : \text{Rnd}(r')$ are irrelevant. As a result, we need to track at most one property of the form $x[t] : \text{Rnd}(r)$ per wire $x[t]$. This property can be computed along with the other properties of $x[t]$, which is trivial for the rules RNDRND and RNDREG. For RNDCOMB, the non-trivial case is when at the properties $x[t] : \text{Rnd}(r)$ and $x[t] : \text{Rnd}(r')$ would hold for $r \neq r'$. In that case, we do not store any Rnd property for $x[t]$. Indeed, this can only cause verification failure through FAILGADGETRANDOM, but as explained previously, this failure replaces a failure of RNDUSEUNIQUE.

Table 1: MATCHI result on study cases. The execution time (on an Intel(R) Core(TM) i7-1165G7 laptop) is compared to the behavioral simulation of one execution of the circuit (simulation is performed with Icarus Verilog 12.0, except for the serialized AES which uses Verilator 5.006. The simulation time includes model compilation.

	Verilog simulation (s)				MATCHI verification (s)			
	$d = 1$	$d = 2$	$d = 3$	$d = 4$	$d = 1$	$d = 2$	$d = 3$	$d = 4$
Round-based AES	53.4	179.4	639.7	2028.8	✓ 11.5	✓ 23.6	✓ 33.0	✓ 45.2
Serialized AES (32-bit)	6.3	306.8	798.5	1735.8	✓ 2.8	✓ 6.0	✓ 12.3	✓ 23.0
Skinny	14.8				✗ 0.4			

We also have a global tracking of all randomness $r = (x, t)$ where $\kappa(x, t) = \text{rnd}$. This structure tracks any u such that $\text{RU}(u)$ or $\text{RL}(u)$ (updated when evaluating the gadget execution corresponding to u), any i such that $\{\text{rnd}_r, \text{sh}_i\} \subset S$ for some $\text{GTE}(S)$ (updated by applying GLITCHTRANSLAKE whenever a wire is evaluated) and any $x'[t']$ with $x' \in \mathcal{O}_C$ such that $\text{rnd}_r \in \text{GD}(x'[t'])$ (updated whenever an output wire is evaluated). Whenever the structure is updated, we check the rules RNDUSEUNIQUE , RNDLEAKSHARE , RNDLEAKRND and FAILOUTPUTRANDOM . In order to not grow this map indefinitely, we garbage-collect this map, removing any entry r if there is no wire in the circuit that is glitch-dependent on r .

6 Tool and case-study

The algorithm of Section 5 is implemented in the tool MATCHI, which takes as input a circuit netlist and a sequence of input signals. The input netlist should be encoded following Yosys JSON netlist format using flip-flops and combinational gates; it may contain arbitrary nesting of modules. The top-level module should have one input clock port and a “kind” signal associated to each other input and output port (to designate at each clock cycle whether the port contains randomness, shares, etc.). In the netlist, some modules are annotated as gadgets. These must have a pipeline architecture and have their input and output ports annotated with a static kind (latency can be checked/computed by MATCHI and masked gates well-formedness is verified). Regarding the sequence of input signals, these should be provided as a VCD file, containing the values of all input wires of the top-level module, as well as the values of the “kind” signals. Based on the input and input kind signals, MATCHI runs the symbolic verification and reports any error. For easier root-cause analysis of the reported issues, MATCHI generates a VCD file where the important properties of each wire are reported.

We used three masked HPC implementations from the literature as study cases: a round-based AES-128 implementation [MCS22], a serialized AES-128 implementation with a 32-bit datapath and an optimized S-box [CC24, CGM⁺24] (the `SubBytes` and `MixColumns` operations are serialized) and a Skinny-384+ implementation with an iterative S-box implementation [VCS22, LL architecture]. While both AES implementations successfully passed verification at the tested orders, the Skinny implementation failed even for the first order.¹¹ A first issue was identified as a programming mistake in the randomness distribution network, resulting in some random bits being used in two gadgets. After fixing this issue, a transition leakage issue was identified in the S-boxes by `FAILDPGE`. This issue is fundamental to the chosen architecture, hence it was not fixed (the obvious fix leads to increased latency, which goes against the goal of this “low-latency” implementation). Finally, we observe that the verification time with MATCHI is generally faster than

¹¹These implementations had been previously verified (the round-based AES with FullVerif, and the serialized implementation with a preliminary version of MATCHI). Since the round-based AES has been written in compliance with the strict FullVerif constraints, the main benefits of MATCHI over FullVerif are formal guarantees and the verification of transition leakage (which does not seem a very likely issue in a round-based implementation in 78 clock cycles).

performing a Verilog simulation of one execution of the circuit.

In these designs, the only non-linear masked components are the HPC2 and HP3 AND gadgets. After annotating these gadgets, we implemented a script to translate a MATCHI-annotated gadget into a SILVER-compatible circuit. Then using SILVER [KSM20], we can verify that the multiplications gadgets (as implemented and annotated) are indeed PINI, which validates the last remaining hypothesis of [Theorem 1](#).

7 Related works

The concepts behind HPC such as the security definitions and composition rules with glitches and transitions have been introduced in [CGLS21, CS21]. As a formal verification for HPC, our composition strategy is very similar, with a few key differences. A first difference lies in our circuit model: while our notion of structural circuit is essentially equivalent to the one of [CS21], our security definitions differ significantly. Our masked circuit definition describes inputs based on the κ function which, besides allowing dynamic data kinds on an input wire, does not relate to the classical notion of “sharing of a variable”, unlike the definition of [CS21]. As a corollary, our definitions of masked circuit and gadget do not require them to be the masked implementation of a function: this is not needed for analyzing security. We can leave the verification of the implementation correctness to standard hardware verification methodologies (testing and/or formal verification).¹² At the composition level, the main difference between [Theorem 1](#) and [CS21] is that we allow freestanding logic gates to manipulate shares. Despite these differences, the core composition ideas in the proof of [Theorem 1](#) (parts 2 and 3) are inspired by the proof of [CS21, Theorem 2].

There exists many tools, such as SILVER [KSM20], Coco [HB21], MaskVerif [BBC⁺19]..., that directly verify the security of masking without relying on the composition of gadgets. The main limitation of these tools is scalability: the verification time grows very quickly with the circuit size and security order (the parameter t of the probing model). Practically, this means that verifying the implementation of entire cryptographic functions with higher-order masking is not feasible for these tools, which motivates the compositional approach. As a compositional verification tool, MATCHI is actually complementary with these tools, since they can be used to verify that the security of the PINI/OPINI gadgets.

FullVerif [CGLS21] is a checker for HPC composition rules. It has the same high-level goal as MATCHI, but uses a different approach: it verifies a set of specific properties that should be satisfied by a correct HPC composition. However, contrary to MATCHI, there is no guarantee that these properties are sufficient to ensure secure composition. Moreover, FullVerif works with a very static model where the kind of every wire in the circuit is statically assigned, all manipulation of shares can only be done in specific gadgets and the control flow is exclusively handled through a dedicated multiplexer gadget.

8 Conclusion

We introduced MATCHI, the first compositional verification tool for masking that can prove the security of large masked hardware circuit. Moreover, thanks to a flexible circuit model, MATCHI is able to verify many circuits architectures, putting few constraints on the hardware designers while increasing confidence in their designs. MATCHI does however face some limitations which could be lifted by future work. An interesting direction is the support of the robust-but-relaxed probing model [MM24] would enable verification of more masking schemes, such as LMDPL [SBHM20]. Another direction is stronger support

¹²As an exception, we verify correctness for public values with INCORRECTOUTPUT: this is easily within our model, and can serve as a check for the input and output kind annotations themselves.

for prime-field masking, which can be useful for implementing post-quantum cryptography algorithms.

Acknowledgements This work has been funded in part by the European Union through the ERC project 101077506 (acronym AMaskZONE). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

- [BBC⁺19] Gilles Barthe, Sonia Belaïd, Gaëtan Cassiers, Pierre-Alain Fouque, Benjamin Grégoire, and François-Xavier Standaert. maskverif: Automated verification of higher-order masking in presence of physical defaults. In Kazue Sako, Steve A. Schneider, and Peter Y. A. Ryan, editors, *Computer Security - ESORICS 2019 - 24th European Symposium on Research in Computer Security, Luxembourg, September 23-27, 2019, Proceedings, Part I*, volume 11735 of *Lecture Notes in Computer Science*, pages 300–318. Springer, 2019.
- [BBD⁺16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong non-interference and type-directed higher-order masking. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 116–129. ACM, 2016.
- [BBP⁺16] Sonia Belaïd, Fabrice Benhamouda, Alain Passelègue, Emmanuel Prouff, Adrian Thillard, and Damien Vergnaud. Randomness complexity of private circuits for multiplication. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 616–648. Springer, 2016.
- [CC24] Momin Charles and Gaëtan Cassiers. SMAesH v2.0.0, 2024. <https://doi.org/10.5281/zenodo.13902486>.
- [CGLS21] Gaëtan Cassiers, Benjamin Grégoire, Itamar Levi, and François-Xavier Standaert. Hardware private circuits: From trivial composition to full verification. *IEEE Trans. Computers*, 70(10):1677–1690, 2021.
- [CGM⁺24] Gaëtan Cassiers, Barbara Gigerl, Stefan Mangard, Charles Momin, and Rishub Nagpal. Compress: Generate small and fast masked pipelined circuits. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):500–529, 2024.
- [CS20] Gaëtan Cassiers and François-Xavier Standaert. Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE Trans. Inf. Forensics Secur.*, 15:2542–2555, 2020.
- [CS21] Gaëtan Cassiers and François-Xavier Standaert. Provably secure hardware masking in the transition- and glitch-robust probing model: Better safe than sorry. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(2):136–158, 2021.

- [CSV24] Gaëtan Cassiers, François-Xavier Standaert, and Corentin Verhamme. Low-latency masked gadgets robust against physical defaults with application to ascon. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):603–633, 2024.
- [FGP⁺18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable masking schemes in the presence of physical defaults & the robust probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):89–120, 2018.
- [HB21] Vedad Hadzic and Roderick Bloem. COCOALMA: A versatile masking verifier. In *Formal Methods in Computer Aided Design, FMCAD 2021, New Haven, CT, USA, October 19-22, 2021*, pages 1–10. IEEE, 2021.
- [IEE06] IEEE. Verilog hardware description language. *IEEE Std 1364-2005*, pages 1–590, 2006.
- [ISW03] Yuval Ishai, Amit Sahai, and David A. Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003.
- [KDT⁺22] Jens-Peter Kaps, William Diehl, Michael Tempelmeier, Ekawat Hom-sirikamol, and Kris Gaj. Hardware api for lightweight cryptography, v1.2. 2022. https://cryptography.gmu.edu/athena/LWC/Implementers_Guide_to_Protected_LWC_HW.pdf.
- [KSM20] David Knichel, Pascal Sasdrich, and Amir Moradi. SILVER - statistical independence and leakage verification. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 787–816. Springer, 2020.
- [MCS22] Charles Momin, Gaëtan Cassiers, and François-Xavier Standaert. Handcrafting: Improving automated masking in hardware with manual optimizations. In Josep Balasch and Colin O’Flynn, editors, *Constructive Side-Channel Analysis and Secure Design - 13th International Workshop, COSADE 2022, Leuven, Belgium, April 11-12, 2022, Proceedings*, volume 13211 of *Lecture Notes in Computer Science*, pages 257–275. Springer, 2022.
- [MM24] Nicolai Müller and Amir Moradi. Robust but relaxed probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(4):451–482, 2024.
- [Nat24] National Institute of Standards and Technology. FIPS 2023: Module-lattice-based key-encapsulation mechanism standard. 2024.
- [SBHM20] Pascal Sasdrich, Begül Bilgin, Michael Hutter, and Mark E. Marson. Low-latency hardware masking with application to AES. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(2):300–326, 2020.
- [VCS22] Corentin Verhamme, Gaëtan Cassiers, and François-Xavier Standaert. Analyzing the leakage resistance of the nist’s lightweight crypto competition’s finalists. In Ileana Buhan and Tobias Schneider, editors, *Smart Card Research and Advanced Applications - 21st International Conference, CARDIS 2022, Birmingham, UK, November 7-9, 2022, Revised Selected Papers*, volume 13820 of *Lecture Notes in Computer Science*, pages 290–308. Springer, 2022.